

ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ ГЕТЕРОГЕННЫХ ДАННЫХ

Архитектура процессоров и технология CUDA

Перцев Дмитрий – 04.10.2026

Содержание лекции



Часть I. История GPGPU

1. История появления GPGPU. От шейдеров к CUDA

Часть II. Параллелизм данных

1. Тренды в процессоростроении. Подходы к проектированию
2. Типы параллелизма. Task vs Data Parallelism

Часть III. Структура программ CUDA C

1. Организация системы. Управление памятью
2. API управления памятью CUDA

Часть IV. Базовые концепции CUDA

1. Ядро, поток, блок потоков, решётка блоков



2. Идентификаторы блоков и потоков

Часть V. Примеры программ

1. Память устройства. Пример: RGB в оттенки серого
2. Примеры: размытие изображения, умножение матриц

ИСТОРИЯ ПОЯВЛЕНИЯ GPGPU

GPGPU & CUDA



- GPU - Graphics Processing Unit
- GPGPU - General-Purpose computing on GPU, вычисления общего вида на GPU
  - Первые GPU от NVIDIA с поддержкой GPGPU - GeForce восьмого поколения, G80 (2006 г.)
- CUDA - Compute Unified Device Architecture
  - Программно-аппаратная архитектура от Nvidia, позволяющая производить вычисления с использованием графических процессоров

С чего все начиналось?



- контроллер дисплея
  - video shifter
  - генератор адресов видео (video address generators)
- видеочип «Pixie» (CDP1861, 1976 год)
  - консоль RCA Studio II
  - совместимый с NTSC видеосигнал с разрешением 62x128 или 64x32
- Monochrome/Color Display Adapter (MDA/CDA)
  - Motorola MC6845 (1978 год)
    - IBM PC (1981 год)
    - Apple II

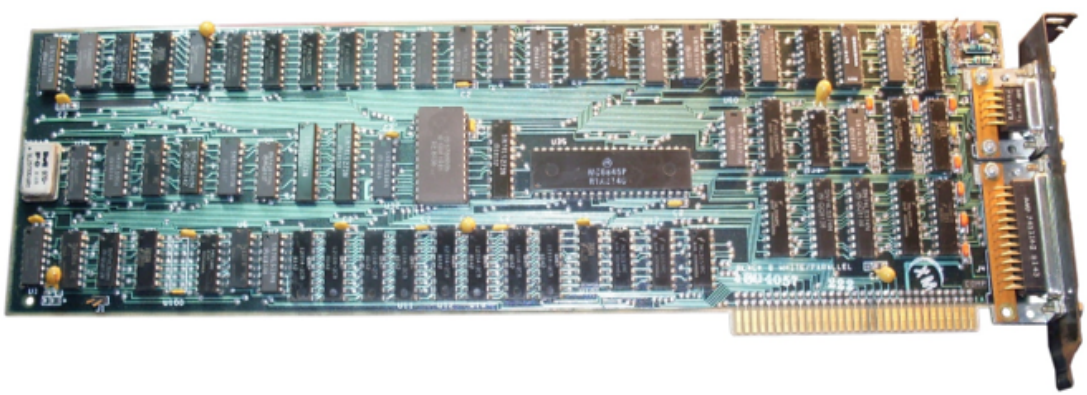


Рис. 1. Контроллер дисплея и видеочип Pixie (CDP1861, 1976)

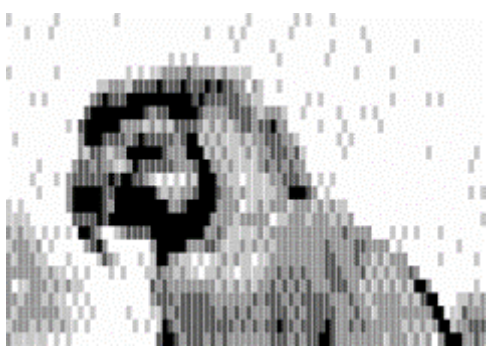


Рис. 2. Консоль RCA Studio II и видеосигнал NTSC

С чего все начиналось?



- Color Graphics Adapter (CGA)
  - 1981 год
  - Motorola 6845
  - ATI CW16800
- CGA 320x200 в 4 цветах, палитра 0 (красный, жёлтый, зелёный, чёрный фон)

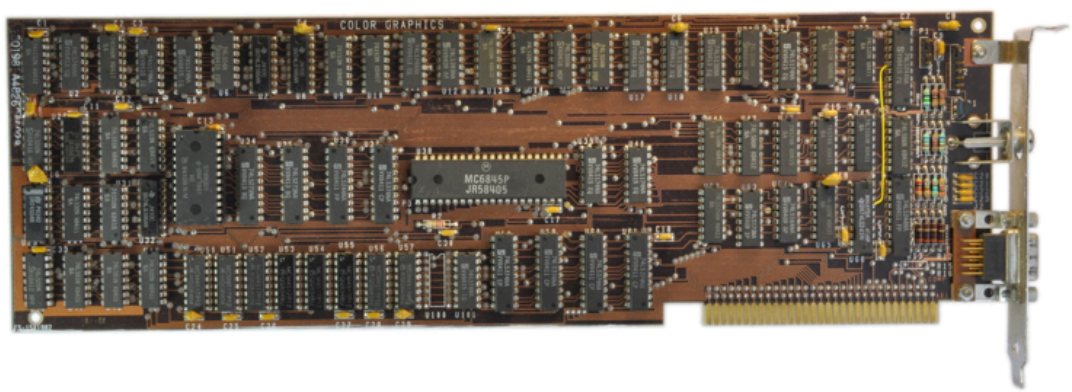


Рис. 3. CGA адаптер (1981)



Рис. 4. CGA 320x200 в 4 цветах

С чего все начиналось?



- Enhanced Graphics Adapter (EGA)
  - 1984 год
  - Motorola 6845
- EGA 640x350 в 16 цветах палитры EGA



Рис. 5. EGA 640x350 в 16 цветах

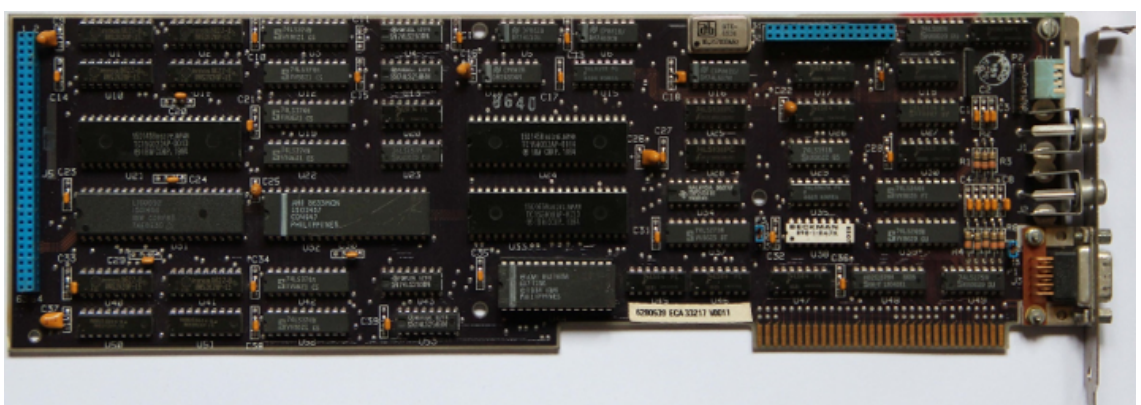


Рис. 6. EGA адаптер (1984)

С чего все начиналось?



- Video Graphics Array (VGA, 1987 год)
  - Chips and Technologies 82c441
  - Video-7 VEGA Deluxe
  - ATI Graphics Solution Plus
  - S3
  - Matrox MAGIC RGB
  - Plantronics Colorplus
  - Paradise Systems PEGA 1
  - Tseng Labs ET3000
  - Cirrus Logic CL-GD400s



Рис. 7. VGA: 640x480 с 16 цветами и 320x200 с 256 цветами

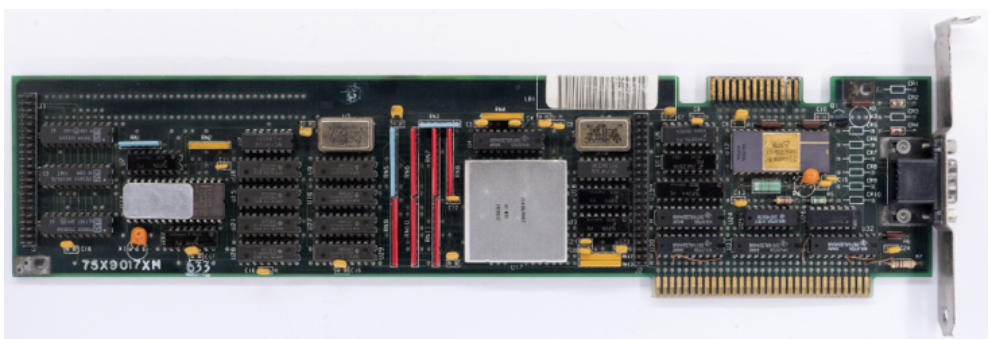


Рис. 8. VGA адаптер (1987)

С чего все начиналось?

- 1992 г. - OpenGL (Silicon Graphics)
- август 1999 г. - появление термина GPU
  - nVidia GeForce 256

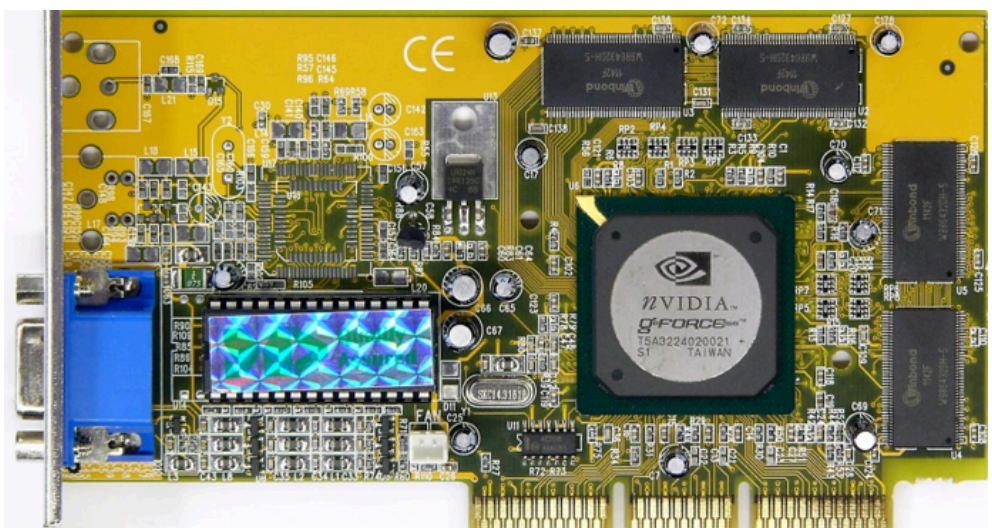


Рис. 9. nVidia GeForce 256 — первый GPU (1999)

С чего все начиналось?

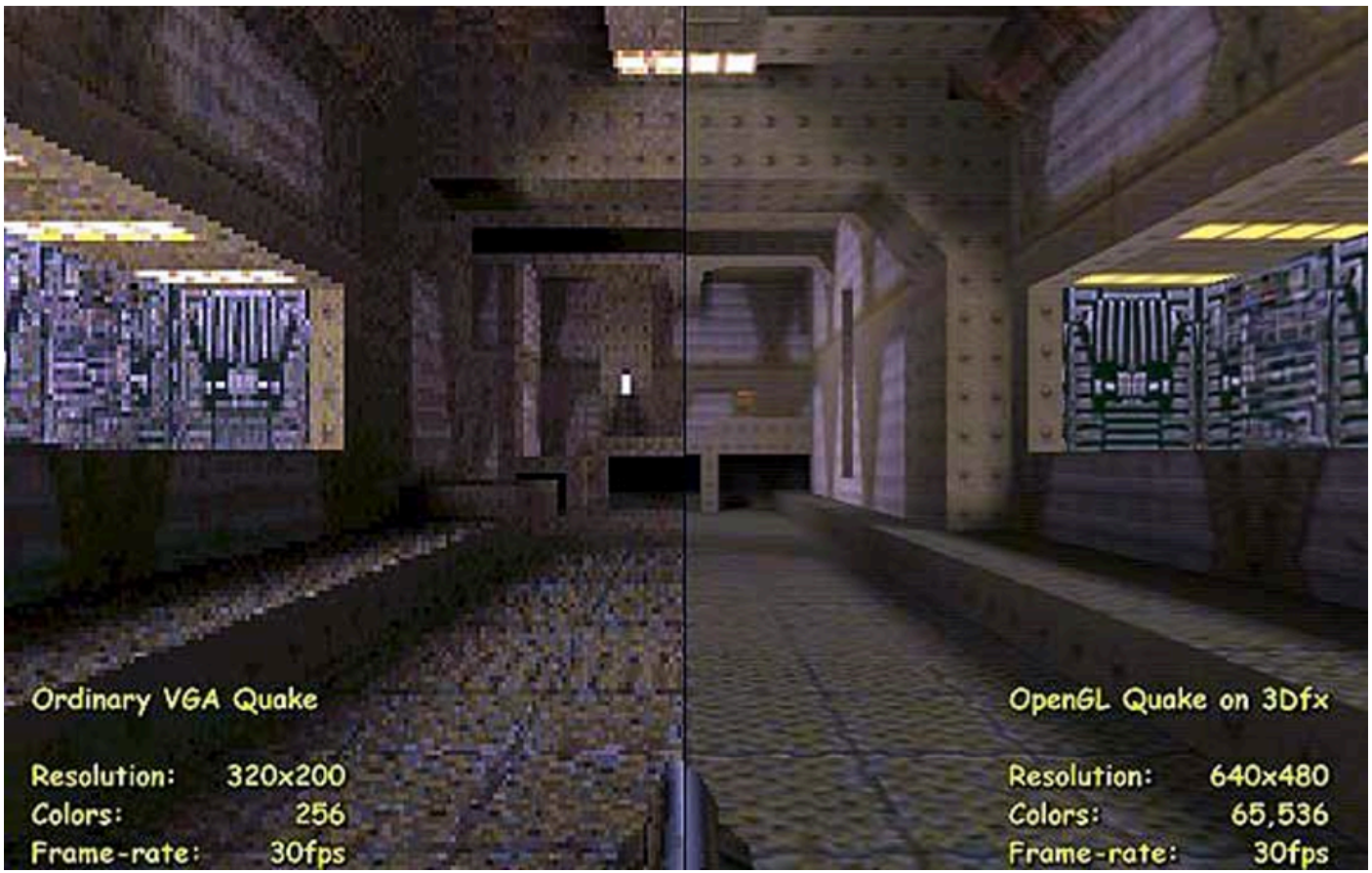


Рис. 10.

С чего все начиналось?

- 2001 г. - GeForce 3
  - Microsoft DirectX 8.0
    - шейдеры

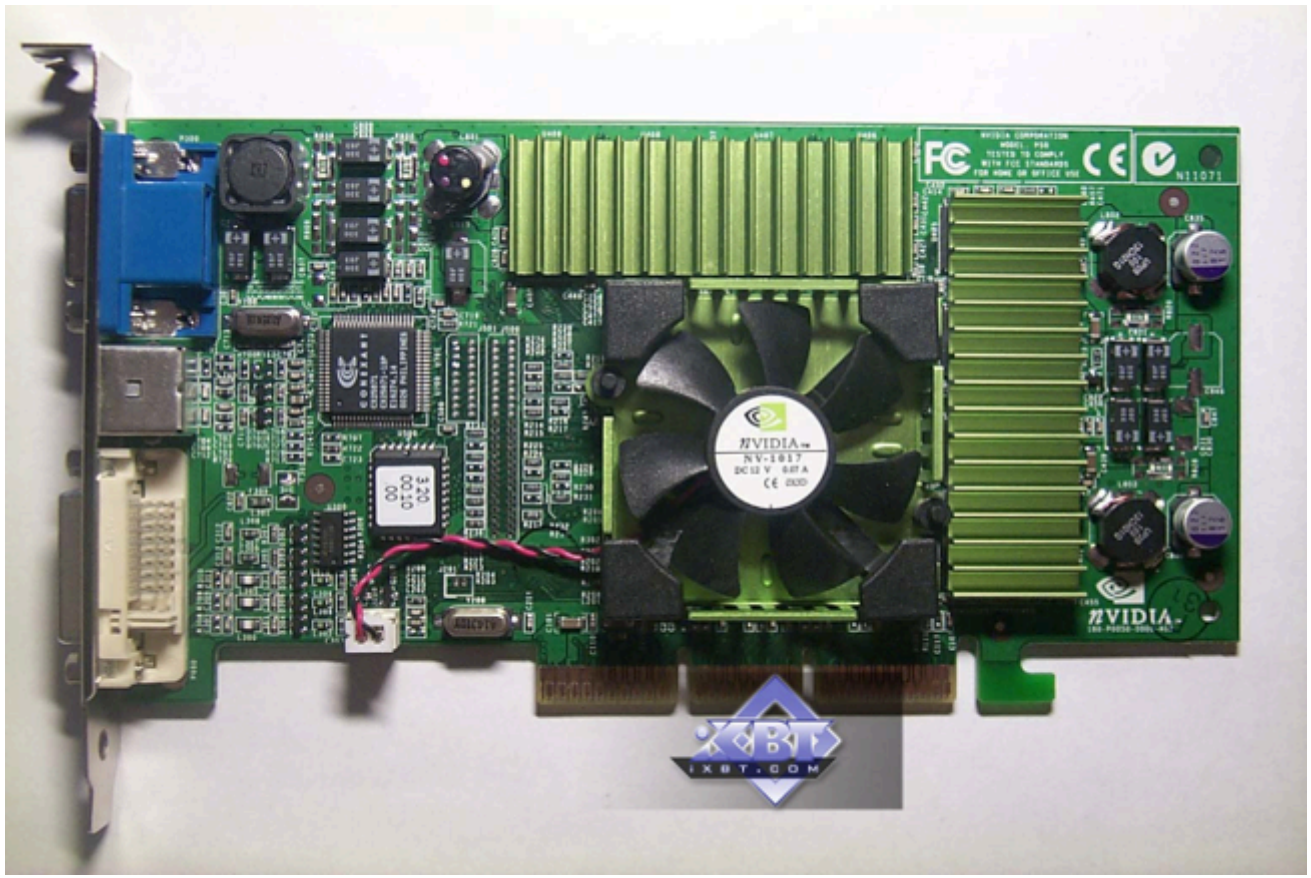


Рис. 11. GeForce 3 и появление шейдеров (2001)

Шейдеры



- специальный ассемблер
- число инструкций: не более 20 команд
- целочисленная арифметика

```
// Hello World! в ассемблере шейдеров
vs.2.0
dcl_position v0
dcl_texcoord v3
m4x4 oPos, v0, c0
mov oT0, v3
```

Шейдеры



- Развитие:
  - Cg от Nvidia
  - HLSL от Microsoft
  - BrookGPU (Стэнфорд)

NVIDIA CUDA

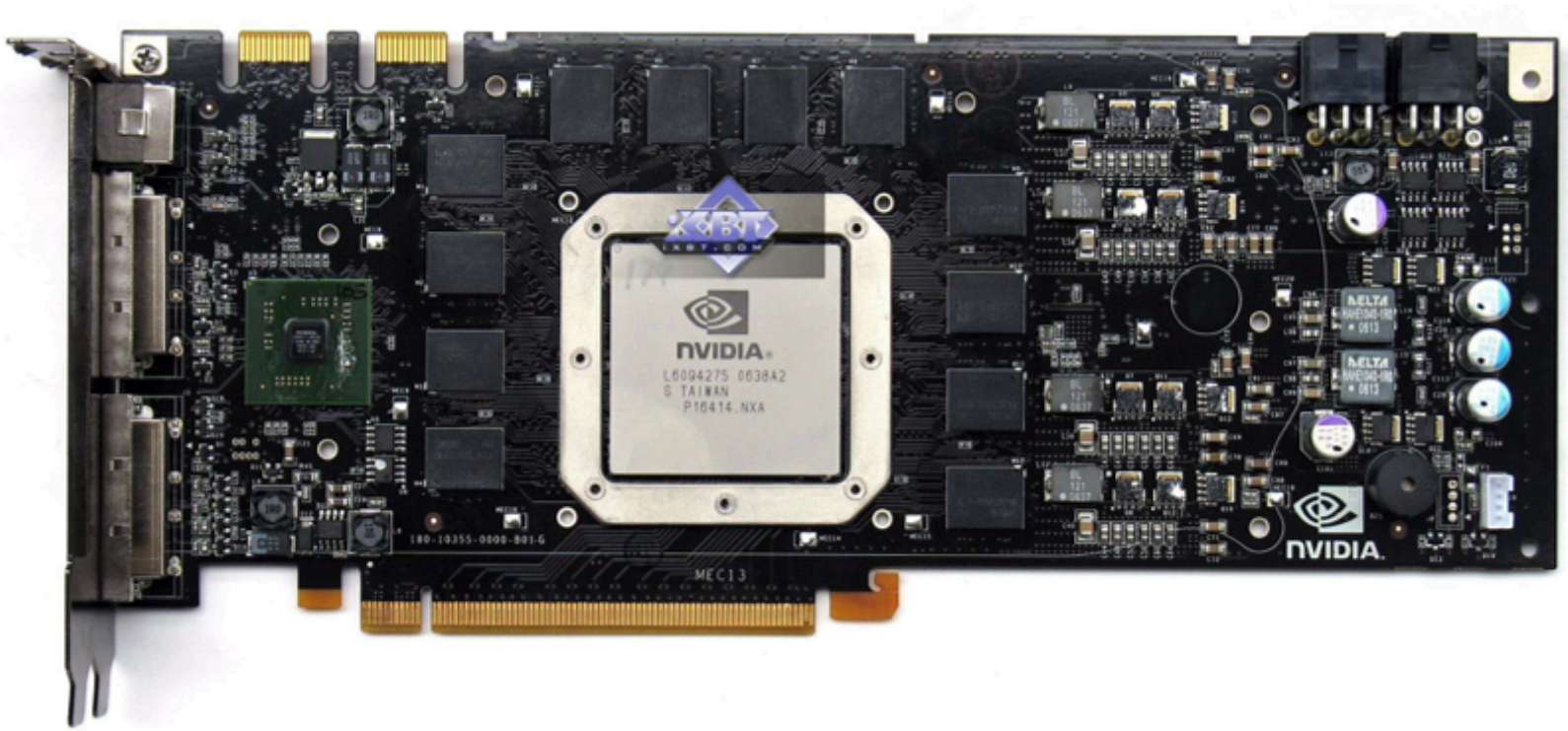


Рис. 12. NVIDIA CUDA 1.0, 2007, NVIDIA G80

Khronos OpenCL



- OpenCL 1.0 - август 2009 г.
- Кросс-платформенная версия

Альтернативные версии



- Intel Xeon Phi (2010 - 2020 годы, снят с производства)
- Microsoft C++ AMP (2012-2022, deprecated)
- AMD FireStream (2006-2012)
  - AMD ROCm (с 2016 года)
- Intel oneAPI / Arc GPU (с 2022 г.)
- Apple Metal / MLX (с 2023 г.)
- Huawei Ascend CANN (с 2020 г.)

Эволюция архитектур NVIDIA

Поколение	Год	Архитектура	Чип	Транзисторов	Память
Volta	2017	sm_70	V100	21.1 млрд	HBM2
Turing	2018	sm_75	T4	18.6 млрд	GDDR6
Ampere	2020	sm_80/86	A100	54.2 млрд	HBM2e
Ada Lovelace	2022	sm_89	L40S	76.3 млрд	GDDR6X
Hopper	2022	sm_90	H100	80 млрд	HBM3
Blackwell	2024	sm_100	B200	208 млрд	HBM3e
Rubin	2026	sm_110	R100	TBD	HBM4

Grace Hopper
Grace Blackwell Superchip

- интеграция CPU + GPU в единый чиплет
  - Grace Hopper GH200 (2023): ARM Neoverse V2 + H100, NVLink-C2C
  - Grace Blackwell GB200 (2024): ARM Neoverse V2 + 2xB200
  - Единое адресное пространство (coherent memory)
  - Пропускная способность NVLink-C2C: 900 ГБ/с

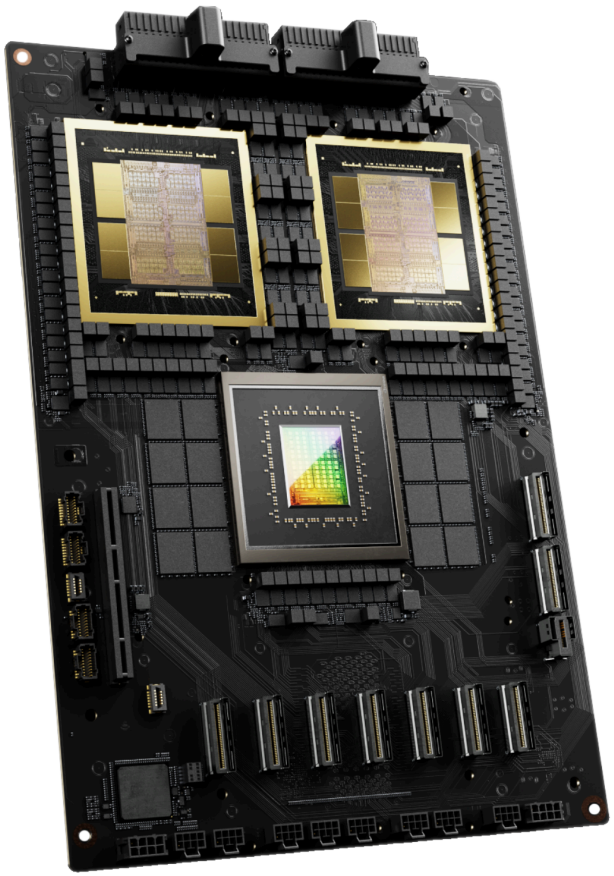


Рис. 13. NVIDIA Grace Blackwell GB200 Superchip

Ускорители в top500

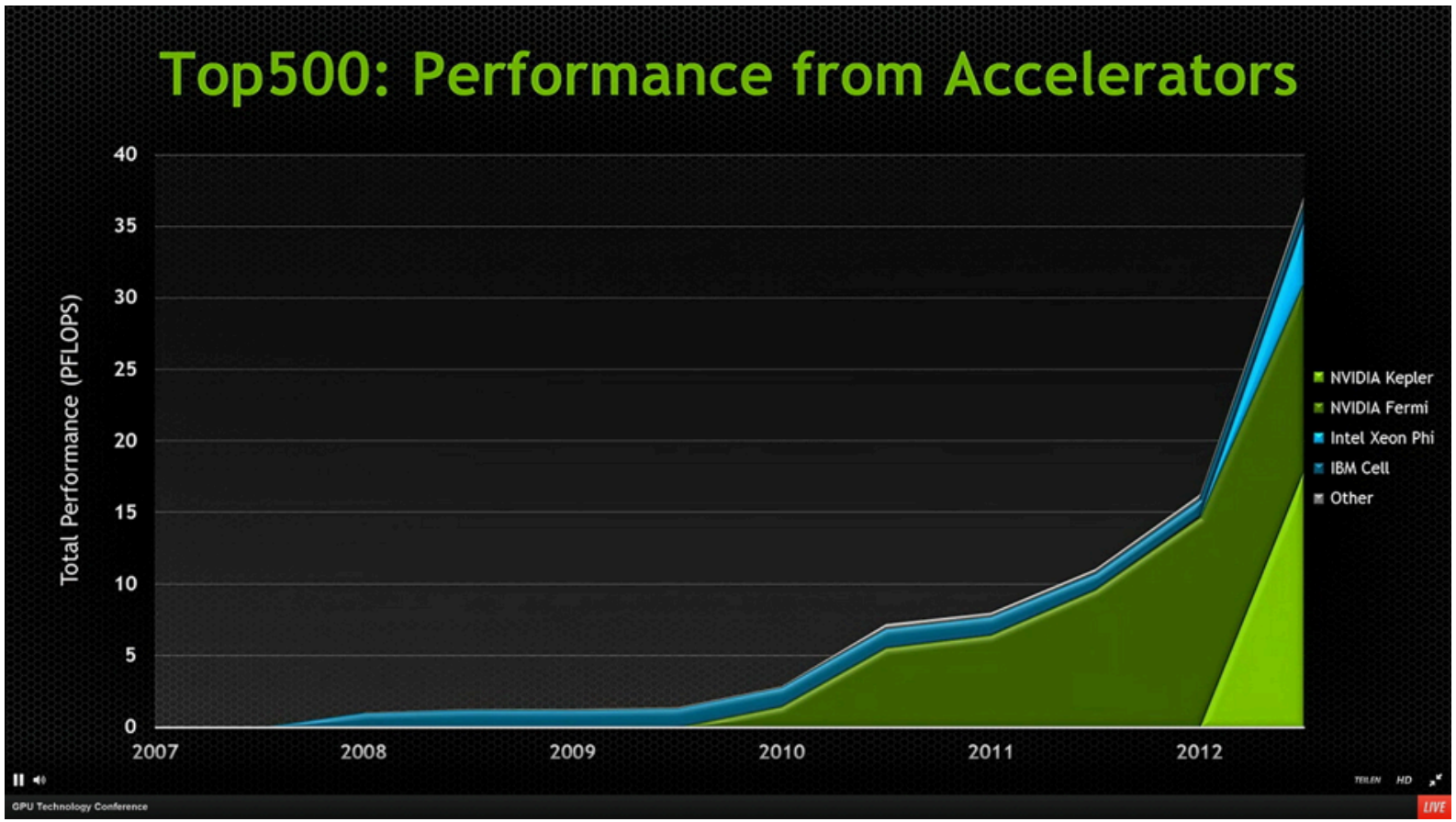


Рис. 14. Доля ускорителей в рейтинге Top500 (2026: 95% систем используют GPU)

Top500 суперкомпьютеров (ноябрь 2025 г.)



- Источник

Rank	Name	Computer	Country	Total Cores	Rpeak (TFlop/s)	Power (kW)
1	Eagle	NVIDIA DGX B200, Grace Blackwell GB200	United States	TBD	TBD	TBD
2	Frontier	HPE Cray EX235a, AMD EPYC 4th Gen, AMD Instinct MI300A	United States	TBD	TBD	TBD

3	El Capitan	HPE Cray EX235n, AMD EPYC, AMD Instinct MI300A	United States	TBD	TBD	TBD
4	Aurora	HPE Cray EX, Intel Xeon 6, Intel Gaudi 3	United States	TBD	TBD	TBD
5	Tianyan 3	Lenovo, AMD EPYC, AMD Instinct MI300X	China	TBD	TBD	TBD

Преимущества GPGPU



- Соотношения:
  - Цена / производительность
  - Производительность / энергопотребление

Green500 (ноябрь 2025 г.)



- Источник

Rank	Name	Computer	Country	Total Cores	Rpeak (TFlop/s)	Power (kW)
1	Eagle	NVIDIA DGX B200, Grace Blackwell GB200	United States	TBD	TBD	TBD
2	Frontier TDS	HPE Cray EX235a, AMD MI300A	United States	TBD	TBD	TBD
3	Perlmutter	HPE Cray EX235n, AMD	United States	TBD	TBD	TBD

		EPYC, NVIDIA A100				

* Лидеры Green500 — системы на базе Grace Blackwell и MI300A с рекордной энергоэффективностью (>100 GFlop/W).

GPU Computing Applications

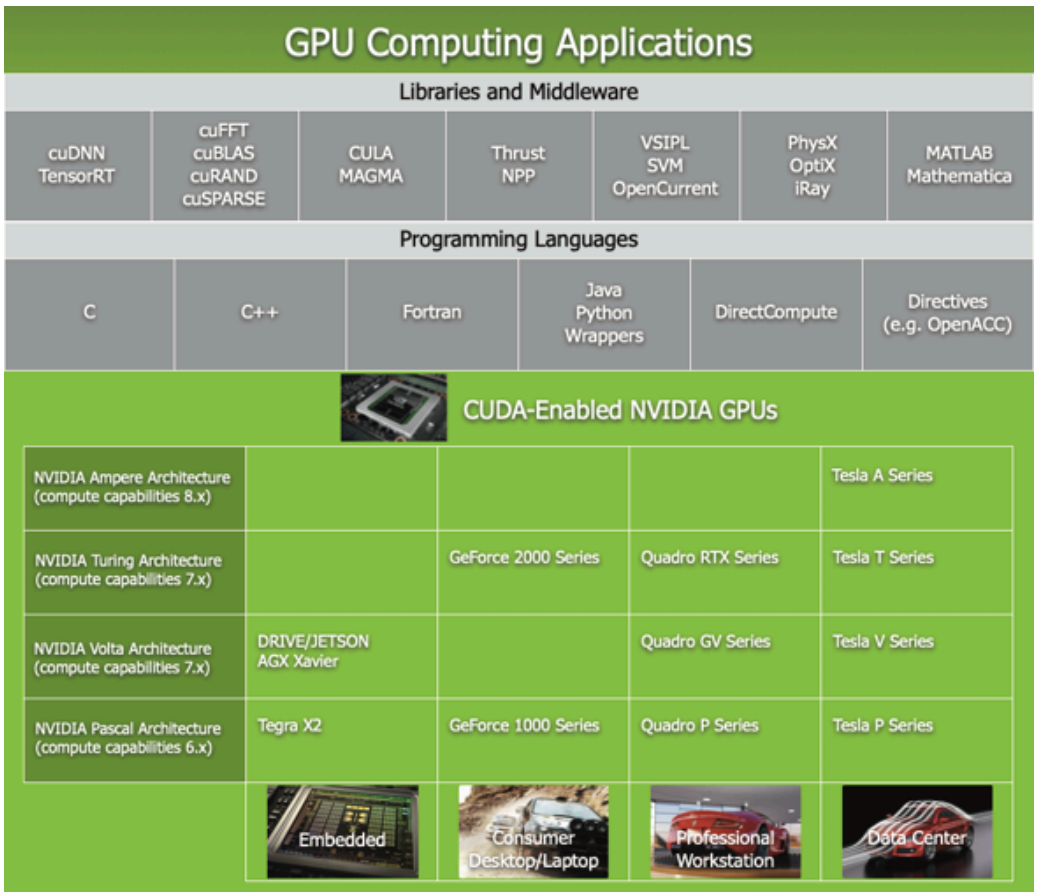


Рис. 15. Области применения GPU-вычислений (Источник)

ТРЕНДЫ В ПРОЦЕССОРОСТРОЕНИИ

Тренды в процессоростроении

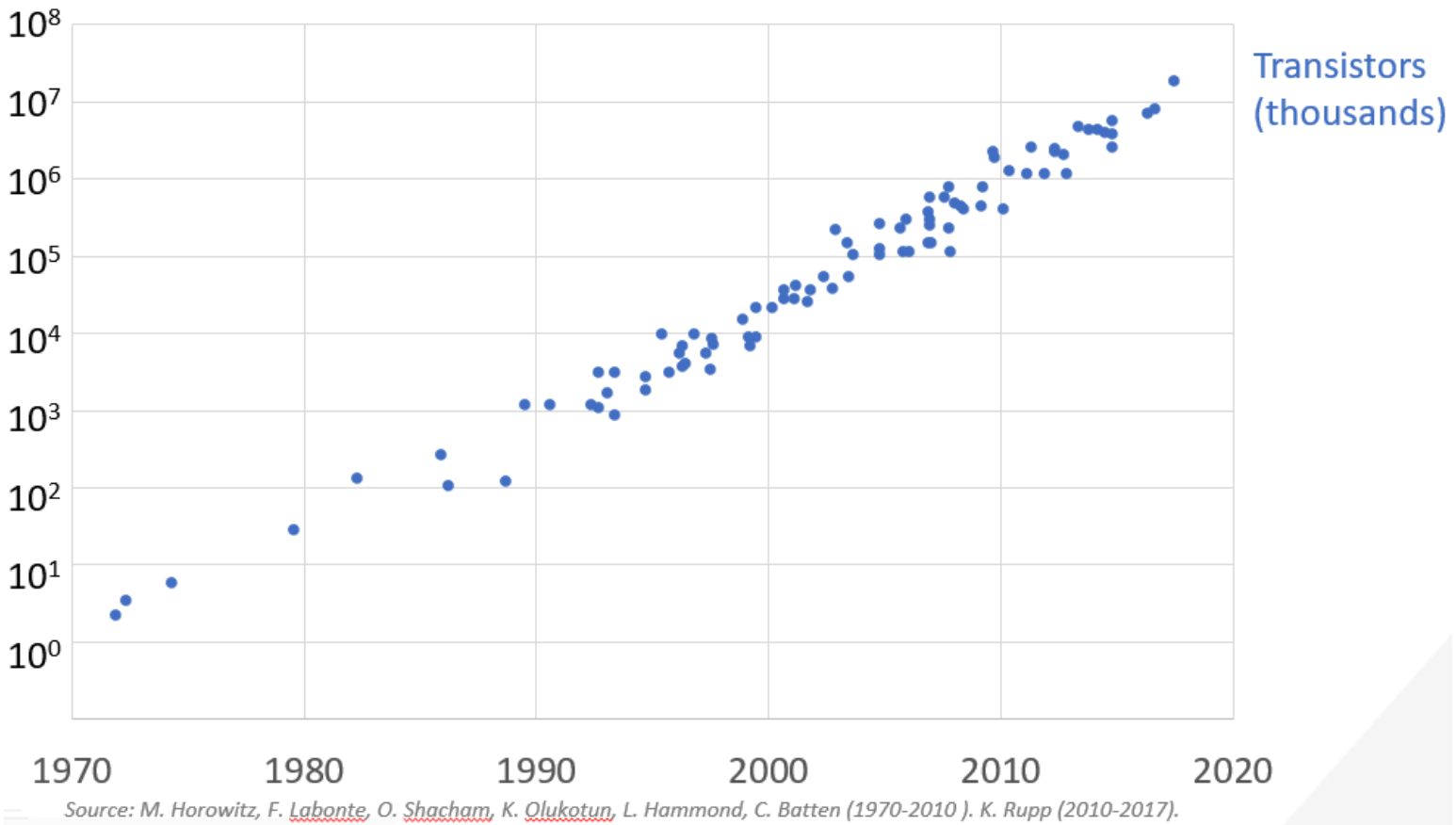


Рис. 16. Закон Мура: удвоение числа транзисторов каждые 18-24 месяца



- **2026**: закон Мура замедлился, переход на чиплетную архитектуру (chiplets)
- NVIDIA Blackwell B200: 2 чипа по 200 млрд транзисторов, соединённых NVLink 5

Тренды в процессоростроении

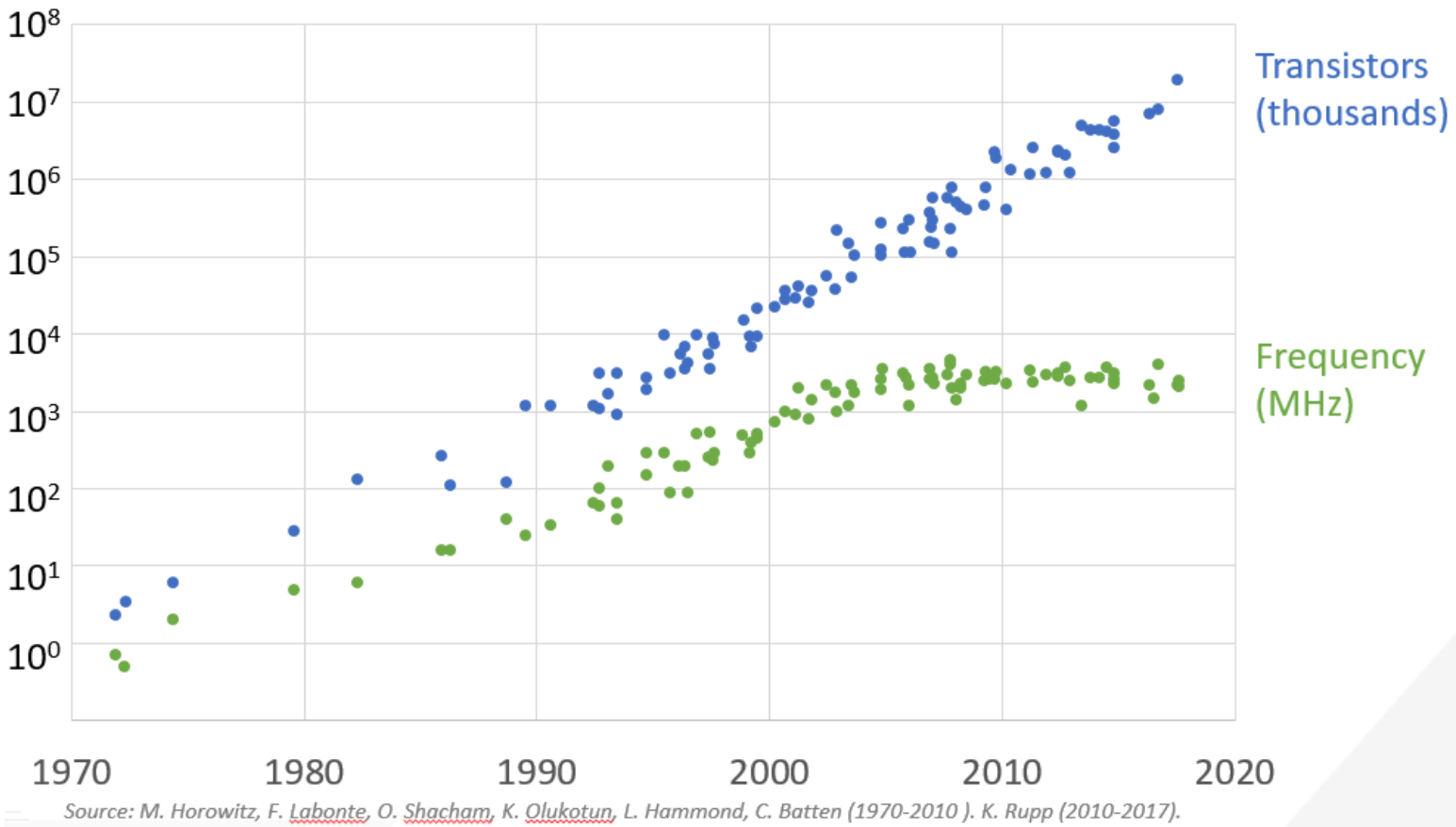


Рис. 17. Рост тактовой частоты процессоров до 2005 года

Тренды в процессоростроении

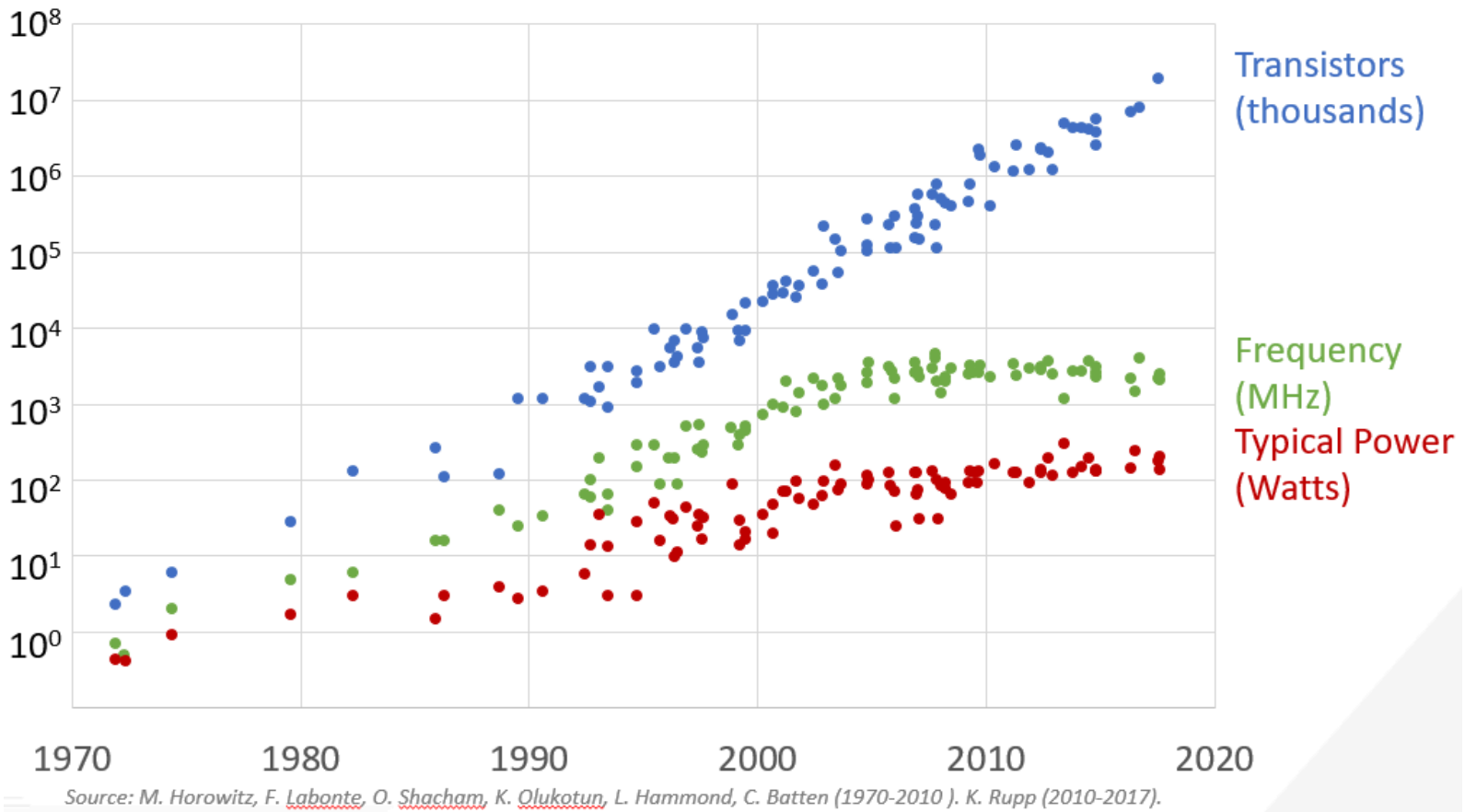


Рис. 18. Тепловой барьер (power wall) около 2005 года

- **в 2026 году**: TDP флагманских GPU достиг 1000-1200W

Тренды в процессоростроении

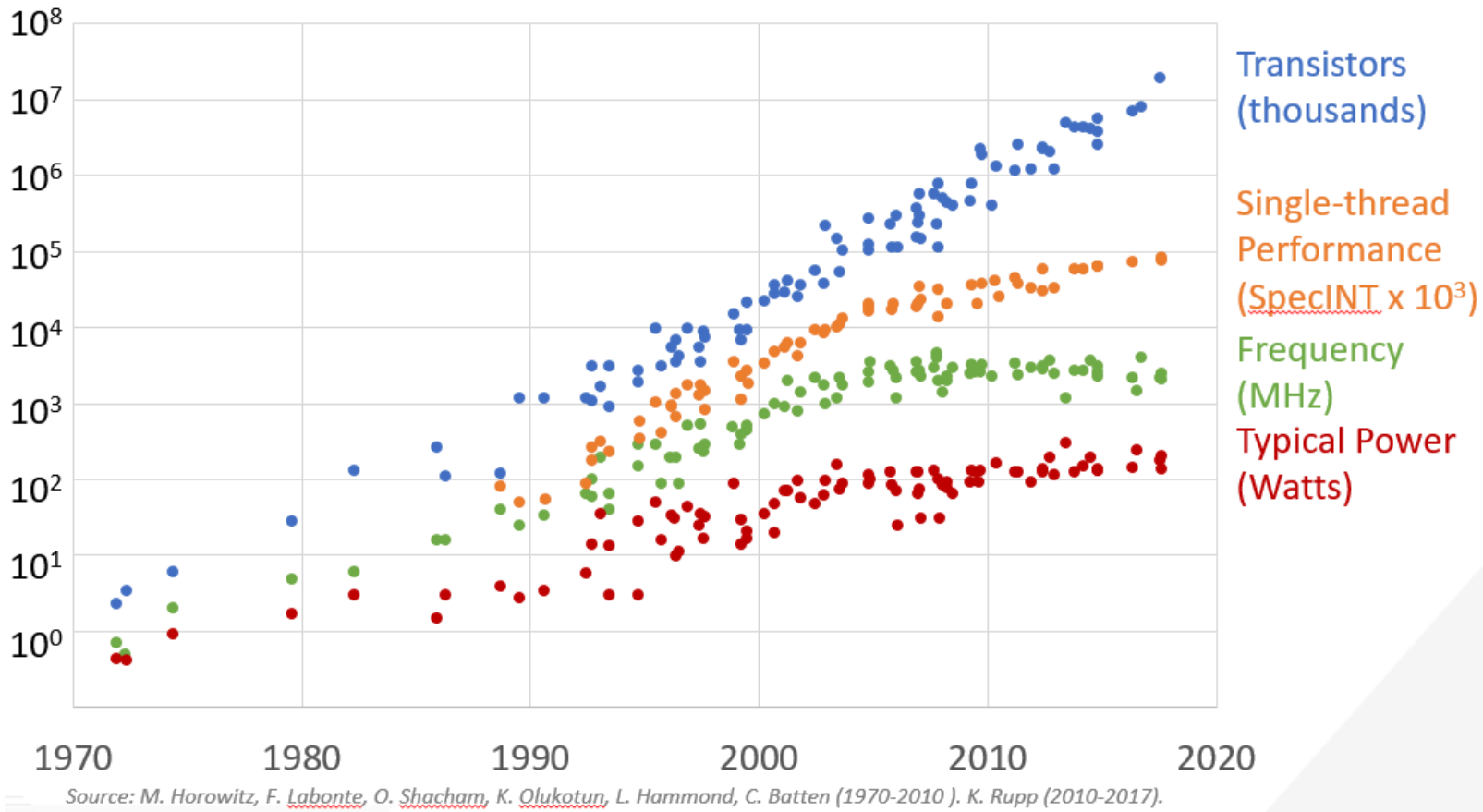


Рис. 19. Стагнация производительности одного потока

Тренды в процессоростроении

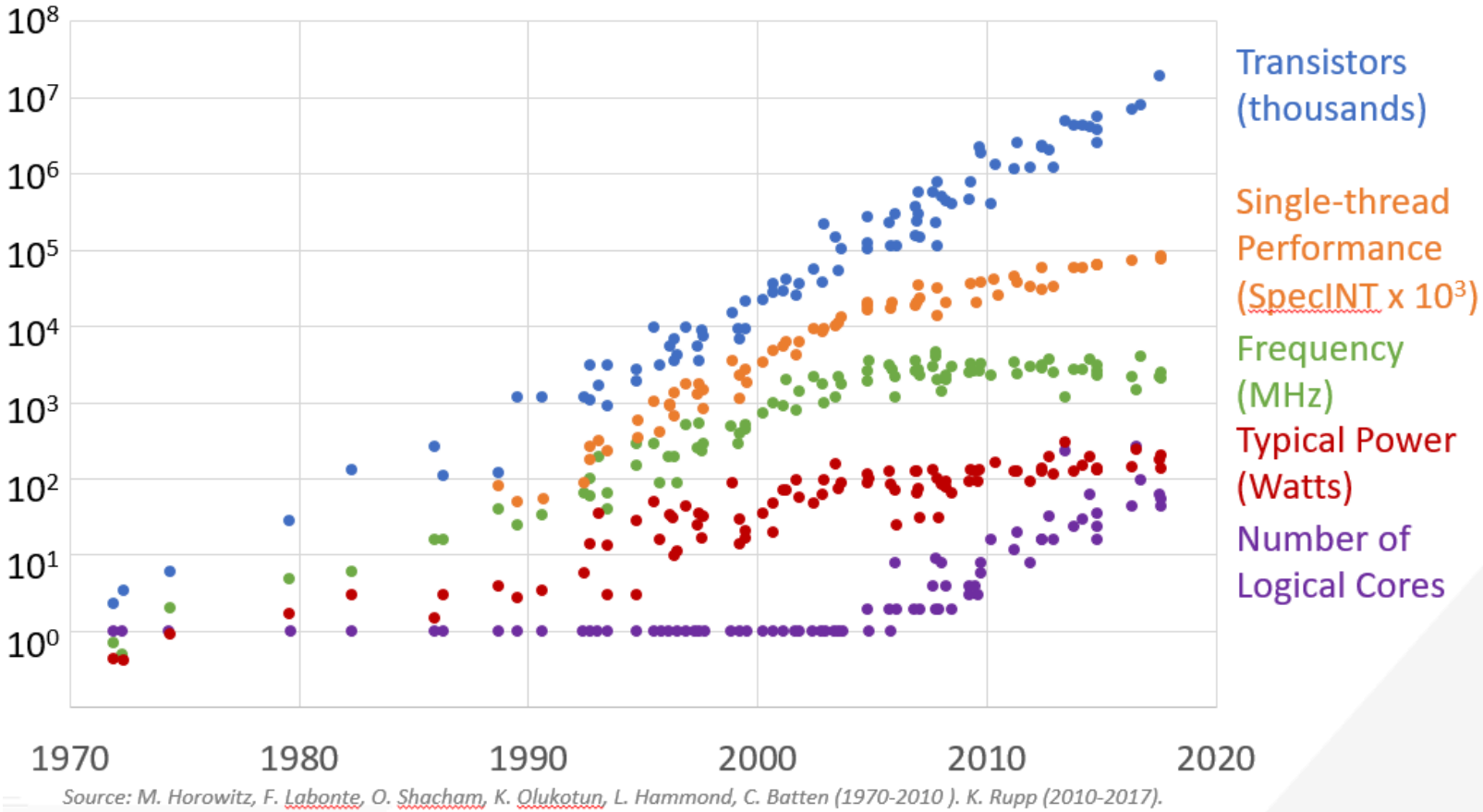


Рис. 20. Рост числа ядер в процессорах

Конец закона Мура? (2020-е)



- «More than Moore» — переход от масштабирования к специализации
  - Чиплеты (chiplets): разделение GPU на несколько кристаллов
  - Advanced Packaging: CoWoS, SoIC, TSMC 3nm/2nm
  - Blackwell B200: 2 кристалла по 104 млрд транзисторов, объединённых NV-HBI (10 ТБ/с)
  - HBM4: 2 ТБ/с на стек, 12-16 Hi

Fermi: Streaming Multiprocessor (SM)

- Потоковый мультипроцессор
- «Единица» построения устройства (как ядро в CPU):
  - 32 скалярных ядра CUDA Core, 1.5 ГГц
  - 2 Warp Scheduler-а
  - Файл регистров, 128 КБ
  - 3 кэша: текстурный, глобальный (L1), константный (uniform)
  - PolyMorphEngine - графический конвейер
  - Текстурные юниты
  - 16 x Special Function Unit (SFU)
  - 16 x Load/Store

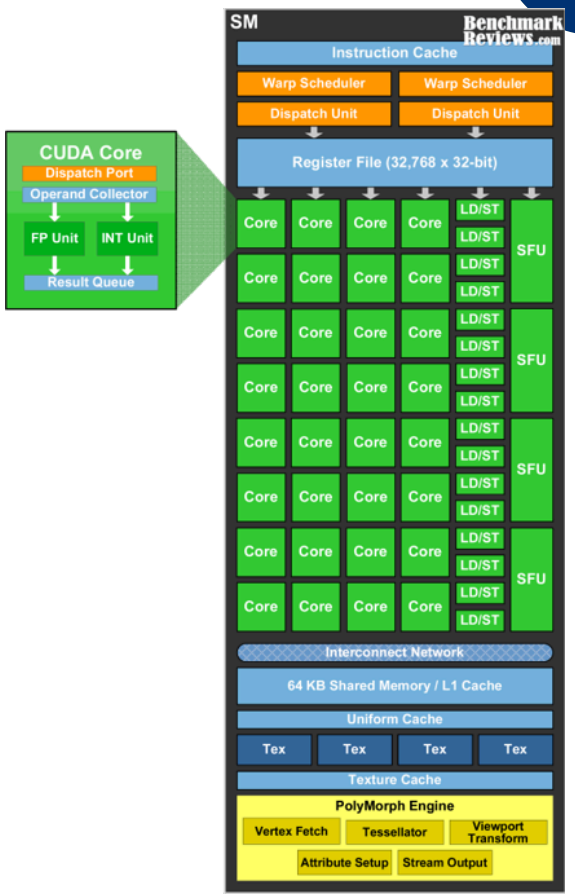


Рис. 21. Архитектура Fermi: Streaming Multiprocessor

Fermi: Чип в максимальной конфигурации

- 16 SM
- 512 ядер CUDA Core
- Кэш L2 758 КБ
- GigaThreadEngine
- Контроллеры памяти DDR5
- Интерфейс PCI



Рис. 22. Fermi: чип в максимальной конфигурации

Blackwell (2024): SM

- 2 чипа в одном корпусе (NV-HBI 10 ТБ/с)
- 192 SM на чип × 2 = 384 SM
- 128 FP32 CUDA Cores на SM
- **49152 FP32** CUDA Cores на весь GPU
- 5-е поколение Tensor Cores
  - FP4, FP6, FP8, FP16, BF16, TF32, INT8, FP4
  - До 20 PFLOPS в AI-задачах
- RAS Engine (аппаратная надёжность)
- Decompression Engine
- Secure AI (TTEE)

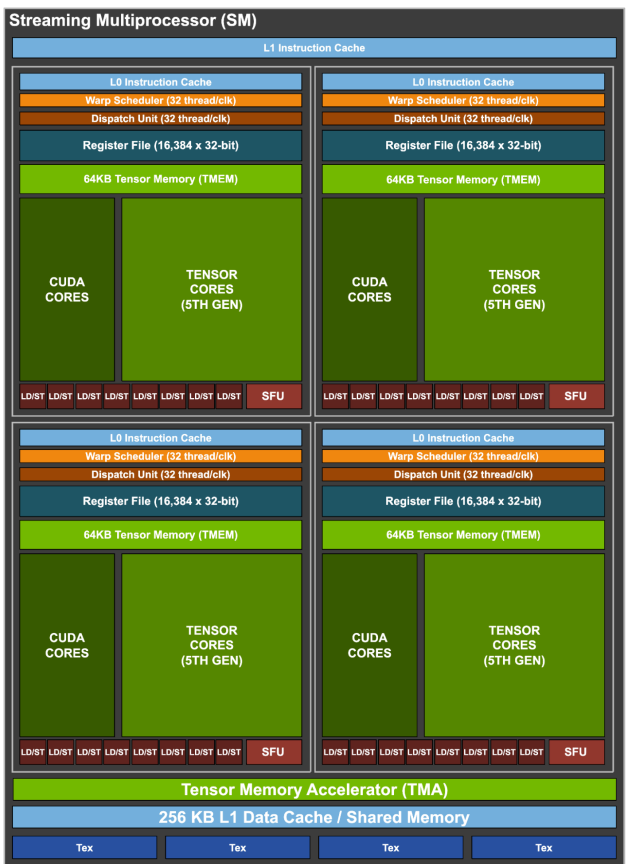


Рис. 23. Blackwell (2024): Streaming Multiprocessor

Подходы к проектированию (Design Approaches)



Рис. 24. Latency-Oriented Design — ориентированный на задержку

Минимизация времени, необходимого для выполнения одной задачи.



Рис. 25. Throughput-Oriented Design — ориентированный на пропускную способность

Максимизация количества задач, которые могут быть выполнены за заданный промежуток времени.

Подходы к проектированию процессоров

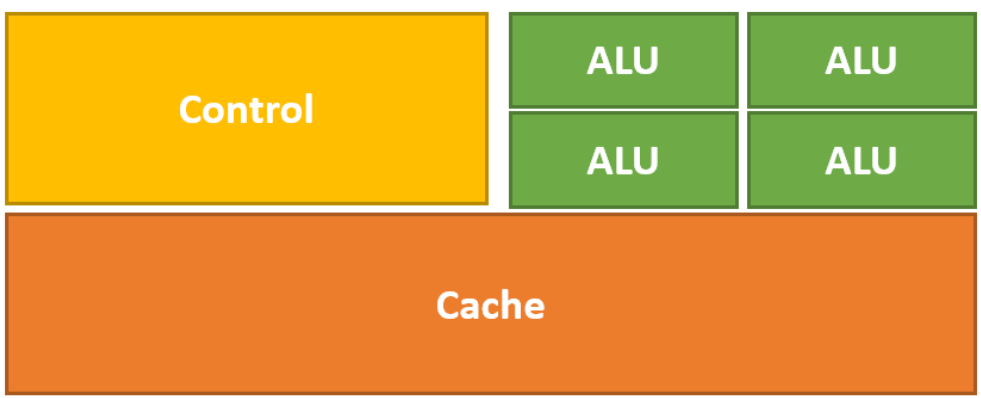


Рис. 26. CPU: Latency-Oriented Design

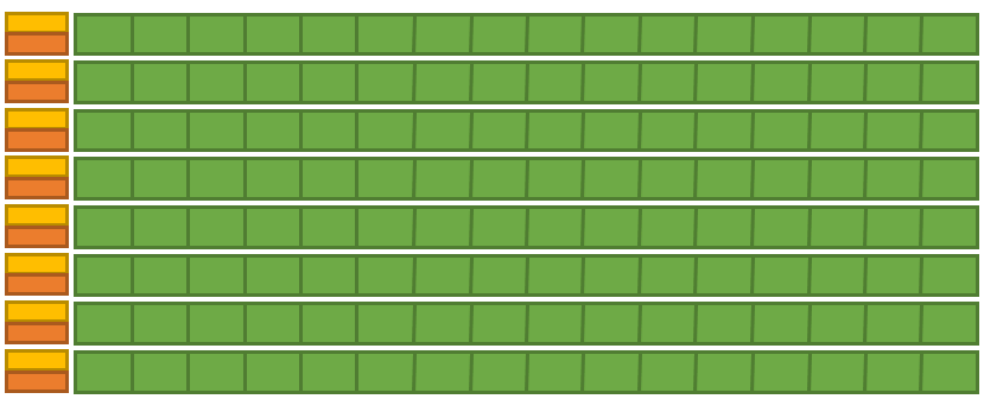


Рис. 27. GPU: Throughput-Oriented Design

Подходы к проектированию процессоров



- Несколько мощных АЛУ
  - Сниженная задержка операций
- Большие кэши
  - Преобразование обращений к памяти с большой задержкой в обращения к кэшу с малой задержкой
- Сложное управление
  - Предсказание ветвлений для снижения конфликтов управления
  - Переадресация данных для снижения конфликтов данных
- Много маленьких АЛУ
  - Большая задержка, высокая пропускная способность
  - Глубокая конвейеризация для дальнейшего увеличения пропускной способности
- Маленькие кэши
  - Больше площади, выделенной под вычисления
- Простое управление
  - Больше площади, выделенной под вычисления



- Умеренная многопоточность для скрытия коротких задержек
- Высокая тактовая частота

- Огромное количество потоков для скрытия очень высокой задержки!
- Умеренная тактовая частота

Проверка ошибок (Error Checking)



- Все вызовы API CUDA возвращают код ошибки `cudaError_t`, который можно использовать для проверки, произошли ли какие-либо ошибки.
  - Пример:

```
cudaError_t err = ...;
if(err != cudaSuccess) {
    printf("Error: %s\n", cudaGetErrorString(err));
    exit(0);
}
```

- Для вызовов ядер можно проверить ошибку, возвращённую `cudaDeviceSynchronize()`, или вызвать следующую функцию API:
  - `cudaError_t cudaGetLastError()`

СТРУКТУРА ПРОГРАММ CUDA C

Сложение векторов (Vector addition)

- Сложение векторов
  - «Hello, World!» для параллельного программирования данных

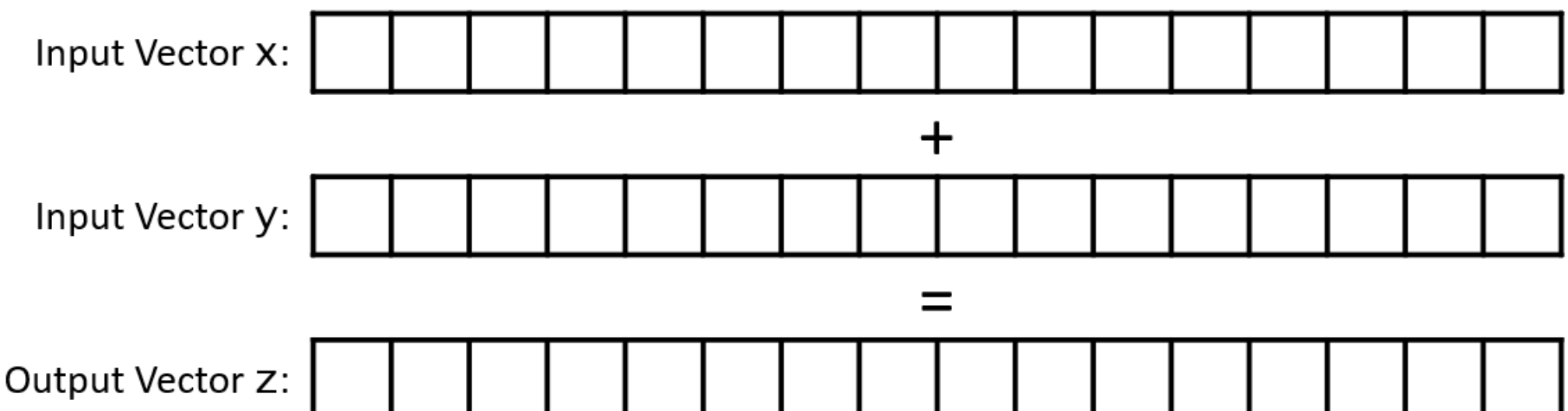


Рис. 28. Сложение векторов — «Hello, World!» параллельного программирования



```
// Последовательный код на C
void vecadd(float* x, float* y, float* z, int N) {
    for(unsigned int i = 0; i < N; ++i) {
        z[i] = x[i] + y[i];
    }
}
```

Организация системы (System Organization)

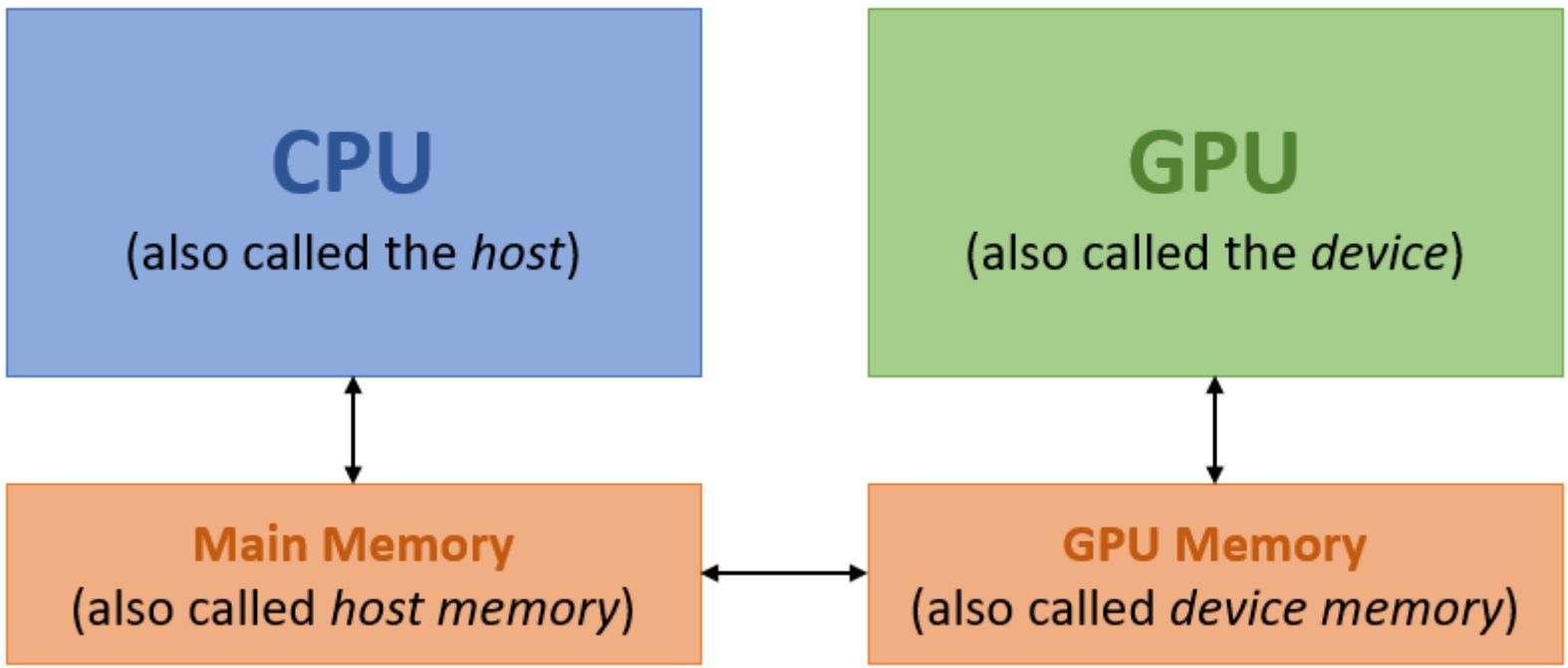


Рис. 29. Организация системы: CPU и GPU имеют раздельную память



- CPU и GPU имеют раздельную память и не могут получать доступ к памяти друг друга
- Необходимо передавать данные между ними
- **CUDA 12.x**: unified memory с page migration (автоматическое перемещение страниц)

Управление памятью (Memory Management)

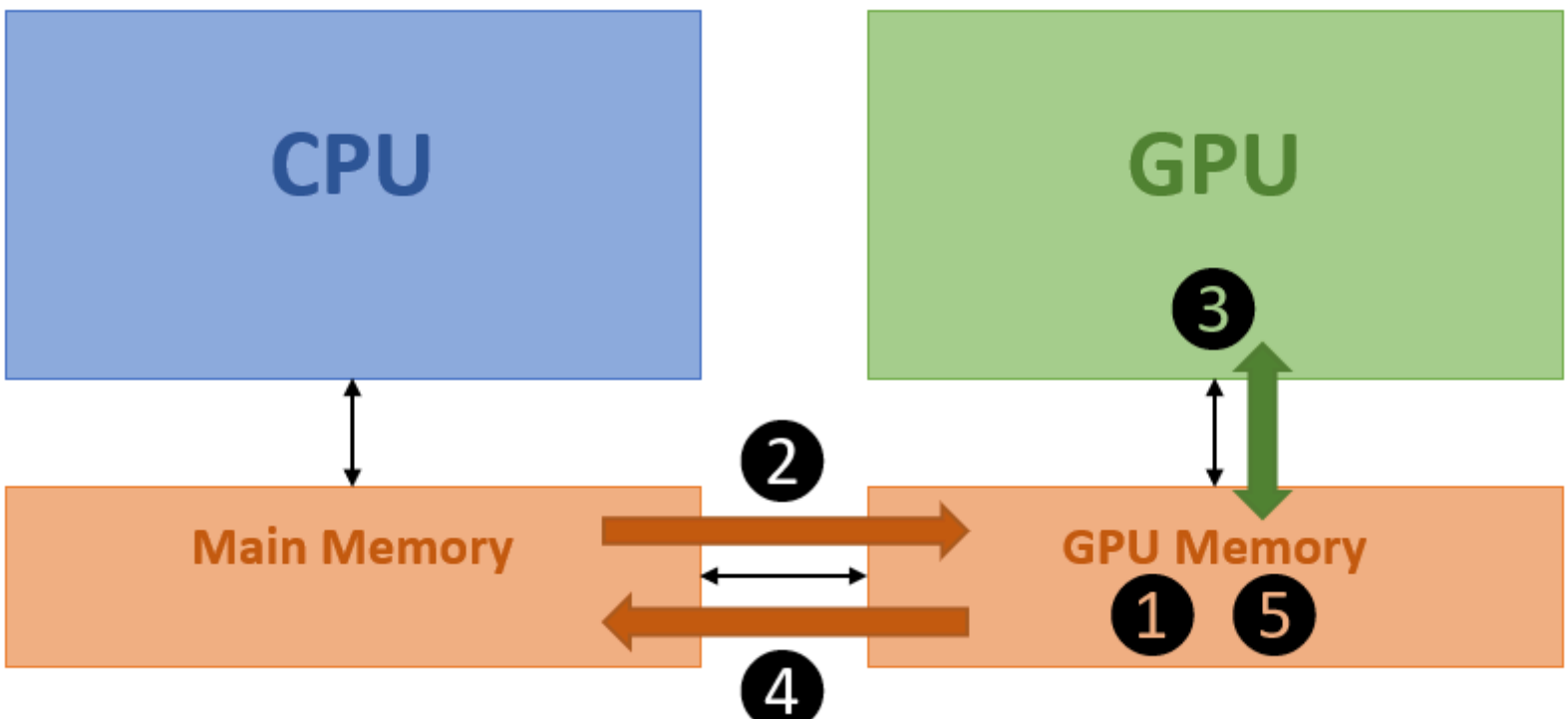


Рис. 30. Управление памятью: этапы работы с GPU



- Выделение памяти GPU (Allocate GPU memory)
- Копирование данных в память GPU (Copy data to GPU memory)
- Выполнение вычислений на GPU (Perform computation on GPU)
- Копирование данных из памяти GPU (Copy data from GPU memory)
- Освобождение памяти GPU (Deallocate GPU memory)

API управления памятью CUDA (базовый)



- Выделение памяти:
  - `cudaError_t cudaMalloc(void **devPtr, size_t size)`
    - `devPtr`: указатель на указатель на выделенную память устройства
    - `size`: запрашиваемый размер выделения в байтах
- Освобождение памяти:
  - `cudaError_t cudaFree(void *devPtr)`
    - `devPtr`: указатель на память устройства для освобождения
- Тип возвращаемого значения: `cudaError_t`
  - Помогает в проверке ошибок (обсуждается позже)

API управления памятью CUDA (Копирование)



- Копирование памяти:
  - `cudaError_t cudaMemcpy(void *dst, const void *src, size_t count, enum cudaMemcpyKind kind)`
    - `dst`: адрес памяти назначения
    - `src`: адрес памяти источника
    - `count`: размер в байтах для копирования
    - `kind`: тип передачи
      - `cudaMemcpyHostToHost`
      - `cudaMemcpyHostToDevice`
      - `cudaMemcpyDeviceToHost`
      - `cudaMemcpyDeviceToDevice`

API управления памятью CUDA 12.x (современный)



- **Асинхронное выделение** (CUDA 11.2+):
  - `cudaMallocAsync(void** ptr, size_t size, cudaStream_t stream)`
  - `cudaFreeAsync(void* ptr, cudaStream_t stream)`
  - использует memory pools для снижения накладных расходов
- **Memory pools**:
  - `cudaMemPoolCreate`, `cudaMemPoolDestroy`
  - повторное использование памяти между запусками
- **Unified Memory** (упрощённое программирование):
  - `cudaMallocManaged` — единое адресное пространство CPU/GPU
  - автоматическая миграция страниц по требованию

Пример кода (базовый)



```
// Выделение памяти GPU
float *x_d, *y_d, *z_d;
cudaMalloc((void**) &x_d, N*sizeof(float));
cudaMalloc((void**) &y_d, N*sizeof(float));
cudaMalloc((void**) &z_d, N*sizeof(float));

// Копирование данных в память GPU
cudaMemcpy(x_d, x, N*sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(y_d, y, N*sizeof(float), cudaMemcpyHostToDevice);

// Выполнение вычислений на GPU
// ...

// Копирование данных из памяти GPU
cudaMemcpy(z, z_d, N*sizeof(float), cudaMemcpyDeviceToHost);

// Освобождение памяти GPU
cudaFree(x_d);
cudaFree(y_d);
cudaFree(z_d);
```

Пример кода (CUDA 12.x, async)



```
// Современный подход с async allocation
cudaStream_t stream;
cudaStreamCreate(&stream);

float *x_d, *y_d, *z_d;
cudaMallocAsync((void**) &x_d, N*sizeof(float), stream);
cudaMallocAsync((void**) &y_d, N*sizeof(float), stream);
cudaMallocAsync((void**) &z_d, N*sizeof(float), stream);

cudaMemcpyAsync(x_d, x, N*sizeof(float), cudaMemcpyHostToDevice, stream);
cudaMemcpyAsync(y_d, y, N*sizeof(float), cudaMemcpyHostToDevice, stream);

vecadd_kernel <<< numBlocks, numThreadsPerBlock, 0, stream >>>
    (x_d, y_d, z_d, N);

cudaMemcpyAsync(z, z_d, N*sizeof(float), cudaMemcpyDeviceToHost, stream);
cudaStreamSynchronize(stream);

cudaFreeAsync(x_d, stream);
cudaFreeAsync(y_d, stream);
```

```
cudaFreeAsync(z_d, stream);
cudaStreamDestroy(stream);
```



Компиляция (Compilation)

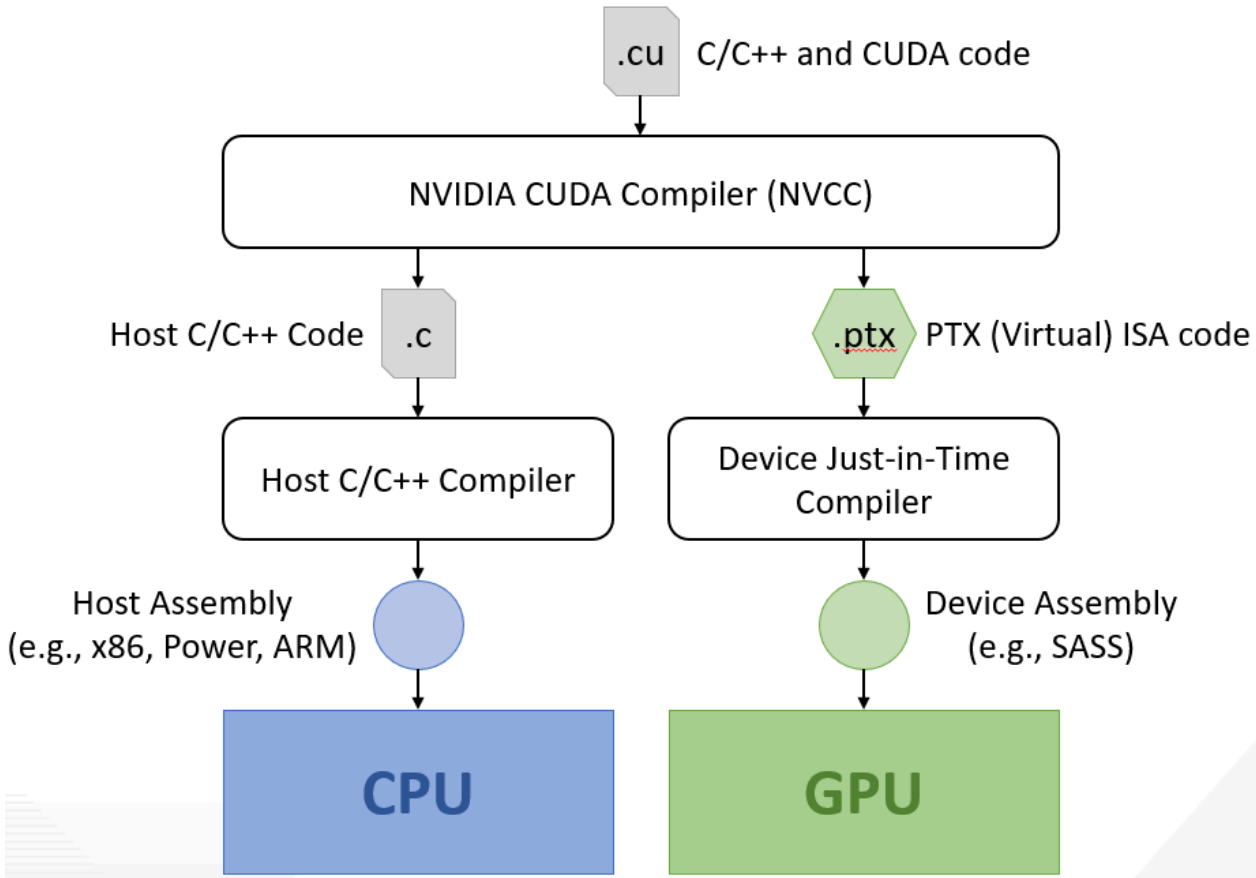


Рис. 31. Компиляция CUDA-программ: host и device код



- nvcc — компилятор CUDA
- Разделяет код на host (CPU) и device (GPU) части
- Device код компилируется в PTX (виртуальная машина) → SASS (нативный код GPU)
- **CUDA 12.x**: поддержка C++17, модулей C++20 (экспериментально)

Объявления функций (Function Declarations)



- Ключевые слова, определяющие, где предназначена для выполнения функция.

Keyword	Callable From	Executed On
`__host__` (default)	Host	Host
`__global__`	Host (or Device)	Device
`__device__`	Device	Device
`__noinline__`	Device	Device (без встраивания)
`__forceinline__`	Device	Device (принудительное встраивание)

Объявления функций (Пример)



```
__host__ __device__ float f(float a, float b) {
    return a + b;
}

void vecadd(float* x, float* y, float* z, int N) {
    for(unsigned int i = 0; i < N; ++i) {
        z[i] = f(x[i], y[i]);
    }
}

__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) {
        z[i] = f(x[i], y[i]);
    }
}
```

Асинхронные вызовы ядер (Asynchronous Kernel Calls)



- По умолчанию вызовы ядер асинхронны.
  - Полезно для перекрытия вычислений на GPU с вычислениями на CPU.
- Используйте следующую функцию API для синхронизации:
  - `cudaError_t cudaDeviceSynchronize()`
  - Блокирует выполнение до тех пор, пока устройство не завершит все предыдущие запрошенные задачи.
- **CUDA 12.x**: CUDA Graphs для снижения накладных расходов на запуск:
  - `cudaGraphCreate`, `cudaGraphLaunch`
  - Запись последовательности операций в граф ⟶ единый запуск

Проверка ошибок (Error Checking)



- Все вызовы API CUDA возвращают код ошибки `cudaError_t`
  - Пример:

```
cudaError_t err = ...;
if(err != cudaSuccess) {
    printf("Error: %s\n", cudaGetErrorString(err));
    exit(0);
}
```

- Для вызовов ядер можно проверить ошибку через `cudaDeviceSynchronize()` или:
  - `cudaError_t cudaGetLastError()`

Проверка ошибок (Error Checking)



- **Макрос для удобной проверки**:

```
#define CUDA_CHECK(call) \
    do { \
        cudaError_t err = call; \
        if(err != cudaSuccess) { \
            printf("CUDA error at %s:%d: %s\n", \
                   __FILE__, __LINE__, cudaGetErrorString(err)); \
            exit(1); \
        } \
    } while(0)

// Использование:
CUDA_CHECK(cudaMalloc(&x_d, N*sizeof(float)));
```

БАЗОВЫЕ КОНЦЕПЦИИ: ЯДРО, ПОТОК, БЛОК ПОТОКОВ, РЕШЁТКА БЛОКОВ

Решётки (Grids)

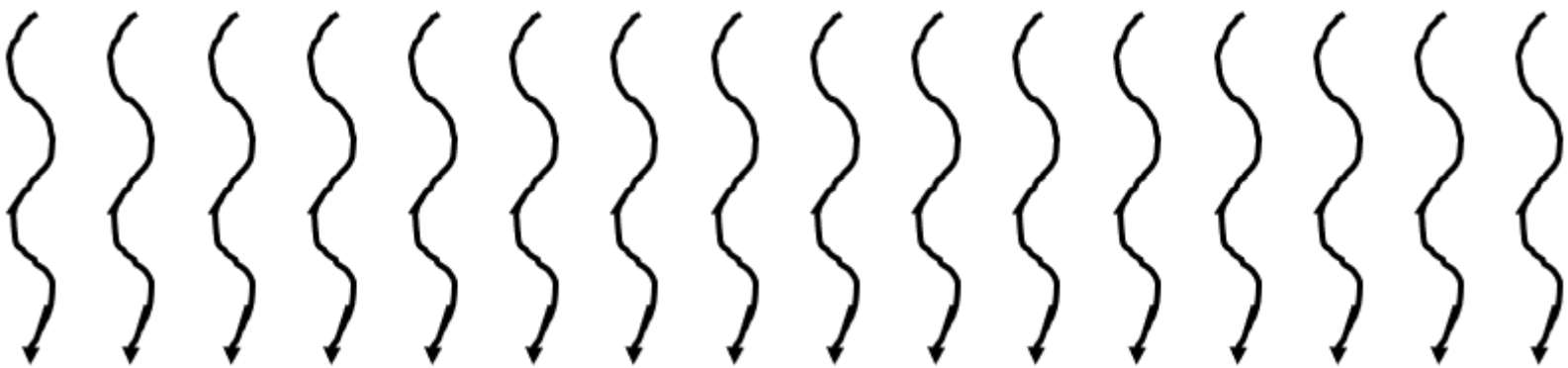


Рис. 32. Решётка (Grid) — массив потоков GPU

- Массив потоков GPU называется решёткой (grid)
- **Hopper+**: решётка может быть организована в Thread Block Clusters

Блоки (Blocks)

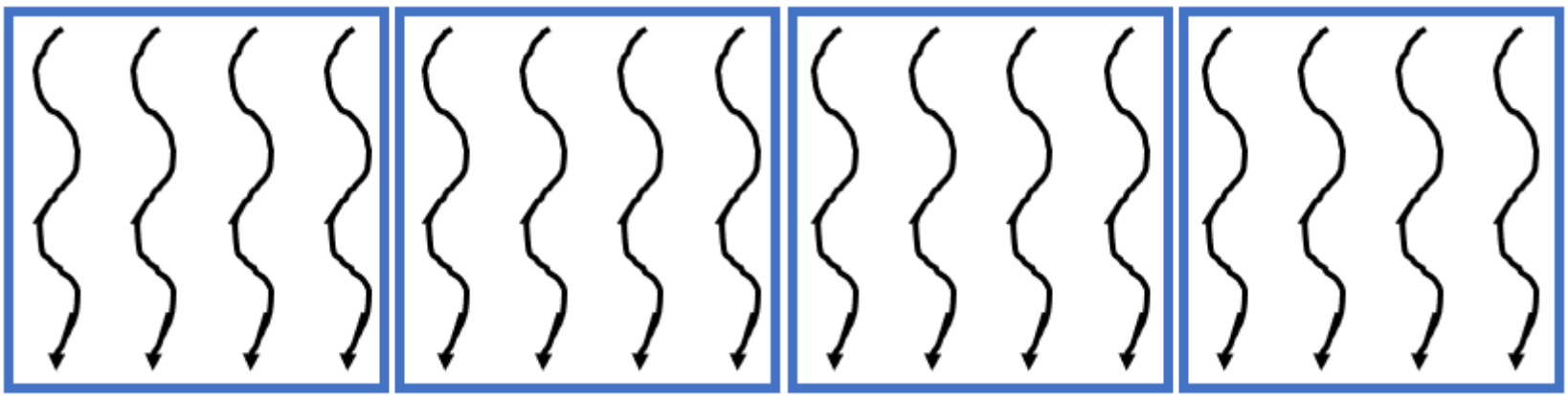


Рис. 33. Блок потоков (Thread Block) — группа потоков в решётке

- Потоки в решётке группируются в блоки потоков (thread blocks)
- **Значение**: потоки в одном блоке могут взаимодействовать способами, недоступными для потоков в разных блоках
  - Барьерная синхронизация (`__syncthreads()`)
  - Разделяемая память (shared memory)

Thread Block Clusters (Hopper, 2022+)



- Группа блоков, гарантированно назначаемых соседним SMs
- Позволяет использовать **распределённую разделяемую память** между блоками кластера
- Прямой доступ к памяти соседних SM без глобальной памяти
- Критично для масштабируемых алгоритмов

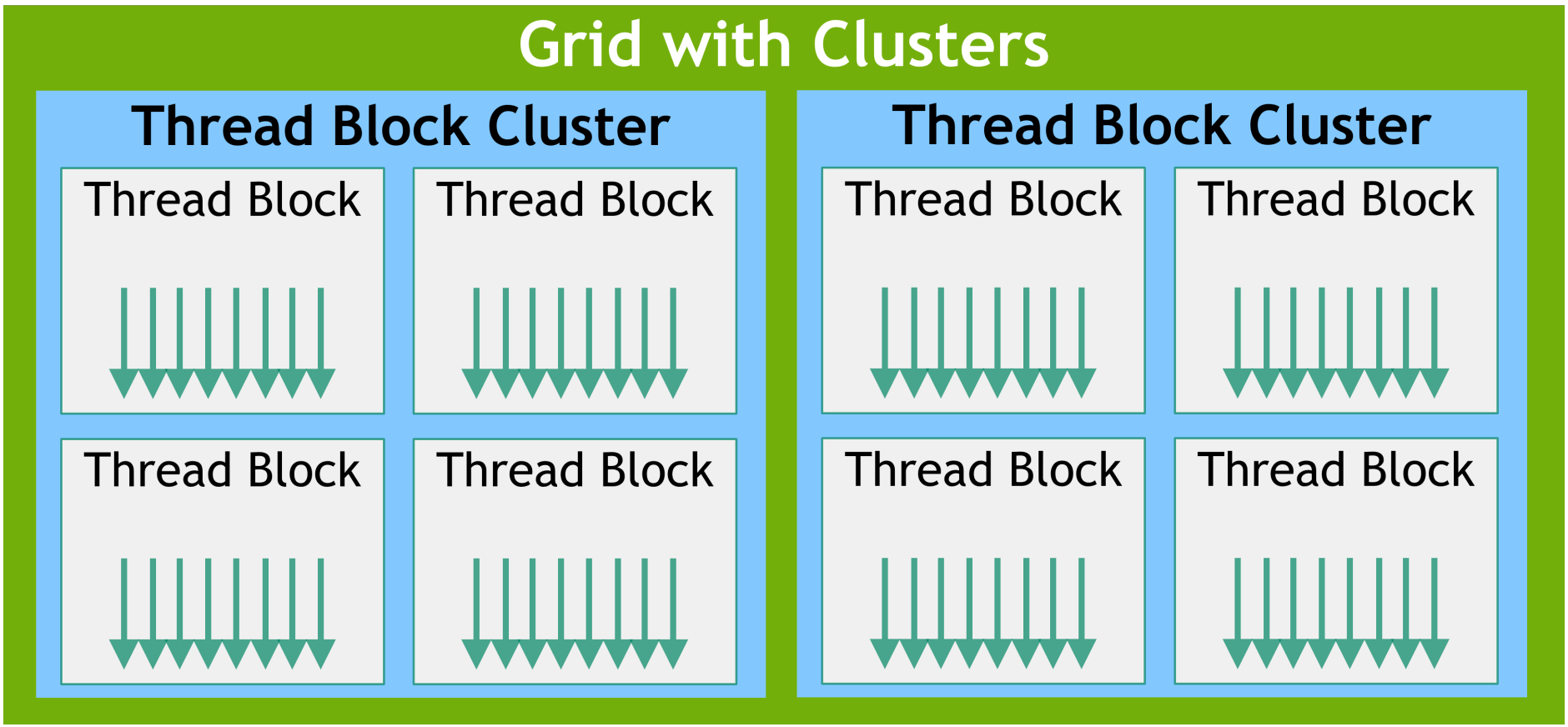


Рис. 34. Thread Block Cluster: группа блоков на соседних SM

Запуск решётки блоков



- Потоки в одной решётке выполняют одну и ту же функцию, известную как **ядро** (kernel).
- Решётка может быть запущена путём вызова ядра и настройки его с соответствующими размерами решётки и блока.
  - Пример:

```
const unsigned int numThreadsPerBlock = 512;
const unsigned int numBlocks = N / numThreadsPerBlock;
vecadd_kernel <<< numBlocks, numThreadsPerBlock >>> (x_d, y_d, z_d, N);
```

Запуск с кластером (Hopper+)



```cpp
// Cluster launch (Hopper+, CUDA 12.0+)
dim3 block(256);
dim3 grid((N + 255) / 256);
dim3 cluster(4);  // 4 блока в кластере

cudaLaunchConfig_t config = {};
config.gridDim = grid;
config.blockDim = block;
config.dynamicSmemBytes = 0;
cudaLaunchAttribute attrs[1];
attrs[0].id = cudaLaunchAttributeClusterDimension;
attrs[0].val.clusterDim = cluster;
config.attrs = attrs;
config.numAttrs = 1;

cudaLaunchKernelEx(&config, my_kernel, args...);
```

Реализация ядра



- Ядро похоже на функцию C/C++.
- Оно предваряется ключевым словом `__global__`, чтобы указать, что это ядро GPU.
- Оно использует специальные ключевые слова для различения разных потоков.

Параллельное сложение векторов в CUDA

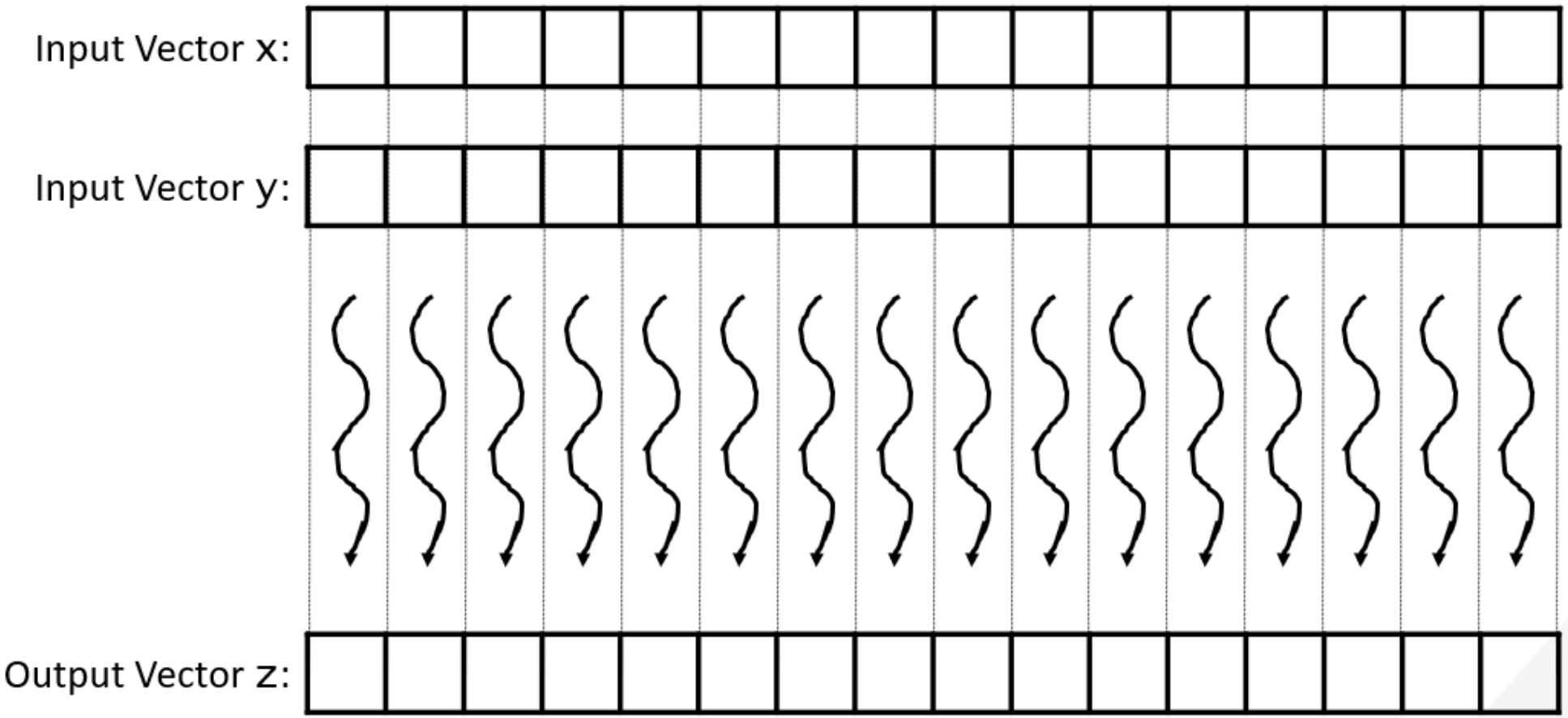


Рис. 35. Параллельное сложение векторов в CUDA

```
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    z[i] = x[i] + y[i];
}
```

Граничные условия (Boundary Conditions)



- Вспомним конфигурации запуска:

```
const unsigned int numThreadsPerBlock = 512;
const unsigned int numBlocks = N / numThreadsPerBlock;
vecadd_kernel <<< numBlocks, numThreadsPerBlock >>> (x_d, y_d, z_d, N);
```

- Если N не кратно `numThreadsPerBlock`, будет запущено меньше потоков, чем требуется.
- Решение: использовать округление вверх для запуска дополнительных потоков, а затем игнорировать потоки за границей.

Выполнение с проверкой границ

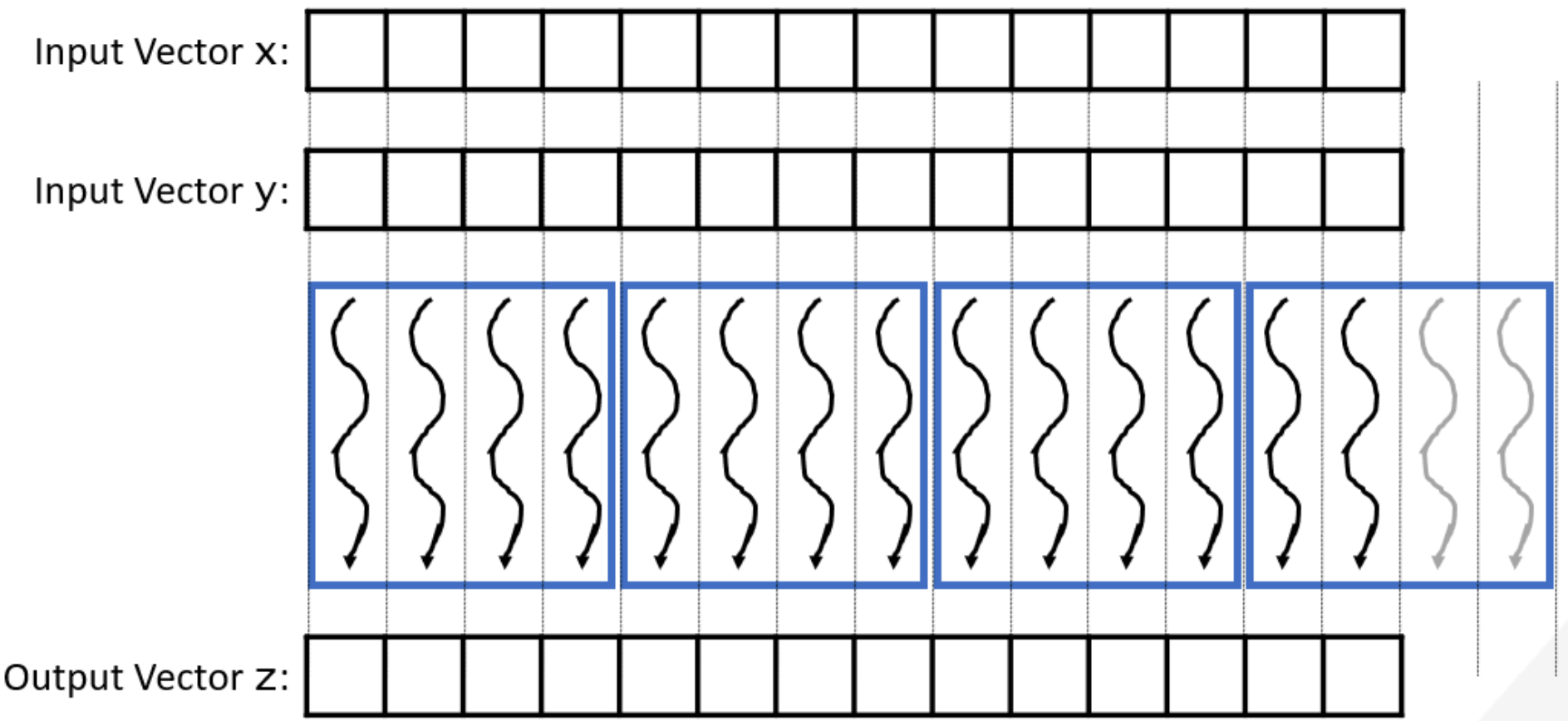


Рис. 36. Выполнение с проверкой границ

Код с проверкой границ



- Код запуска:

```
const unsigned int numBlocks = (N + numThreadsPerBlock - 1) / numThreadsPerBlock;
```

- Код ядра:

```
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if(i < N) {
        z[i] = x[i] + y[i];
    }
}
```



ИДЕНТИФИКАТОРЫ БЛОКОВ И ПОТОКОВ

Размерность решётки (Grid Dimension)

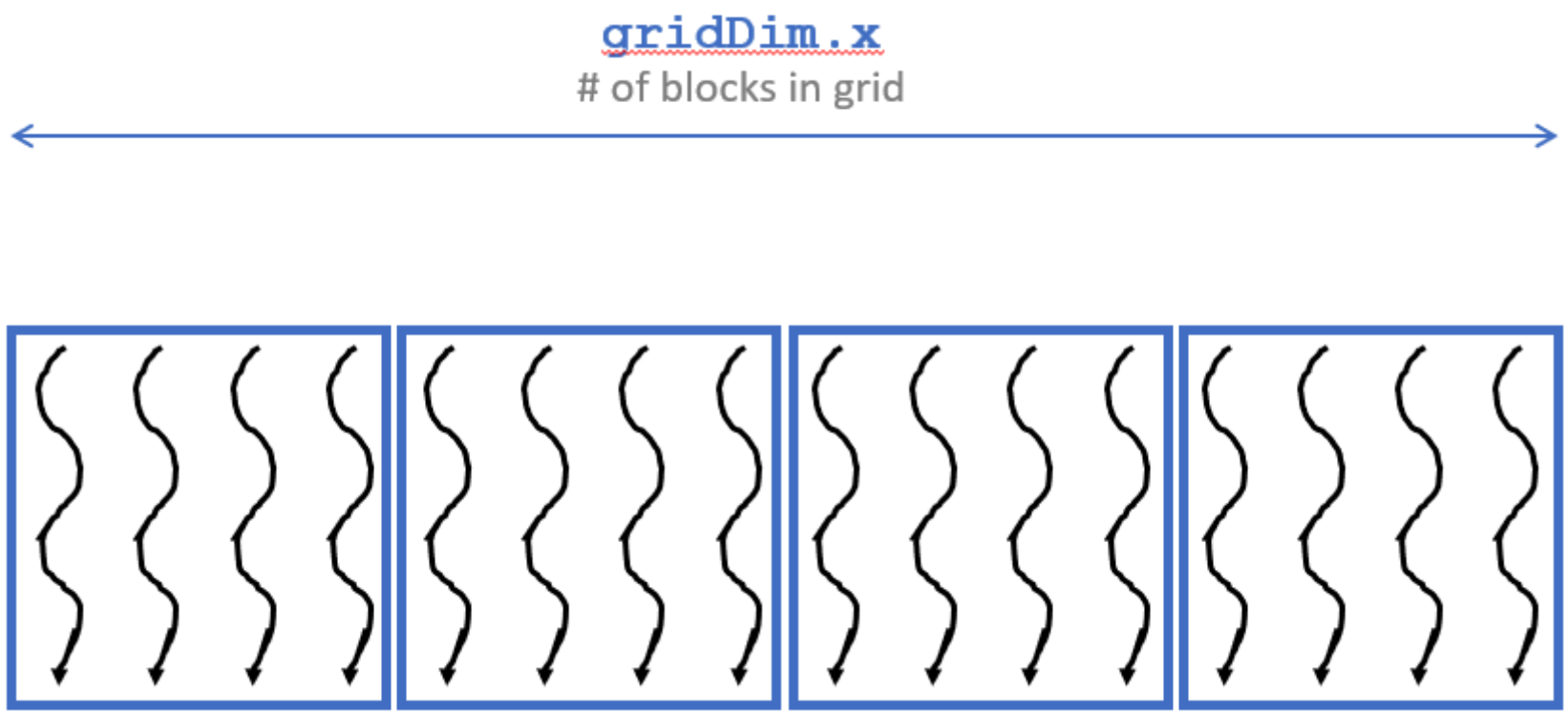


Рис. 37. gridDim — размерность решётки

Индекс блока (Block Index)

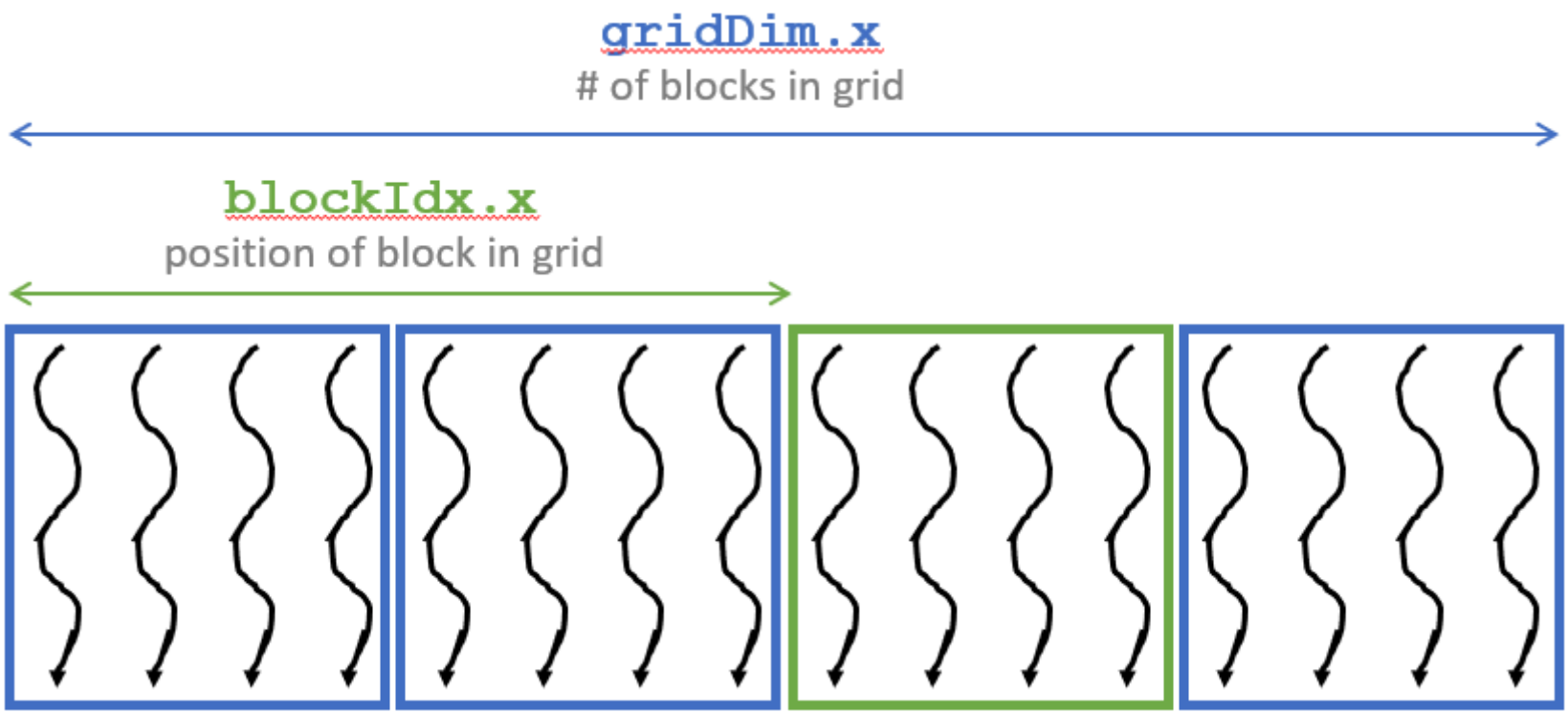


Рис. 38. blockIdx — индекс блока в решётке

Размерность блока (Block Dimension)

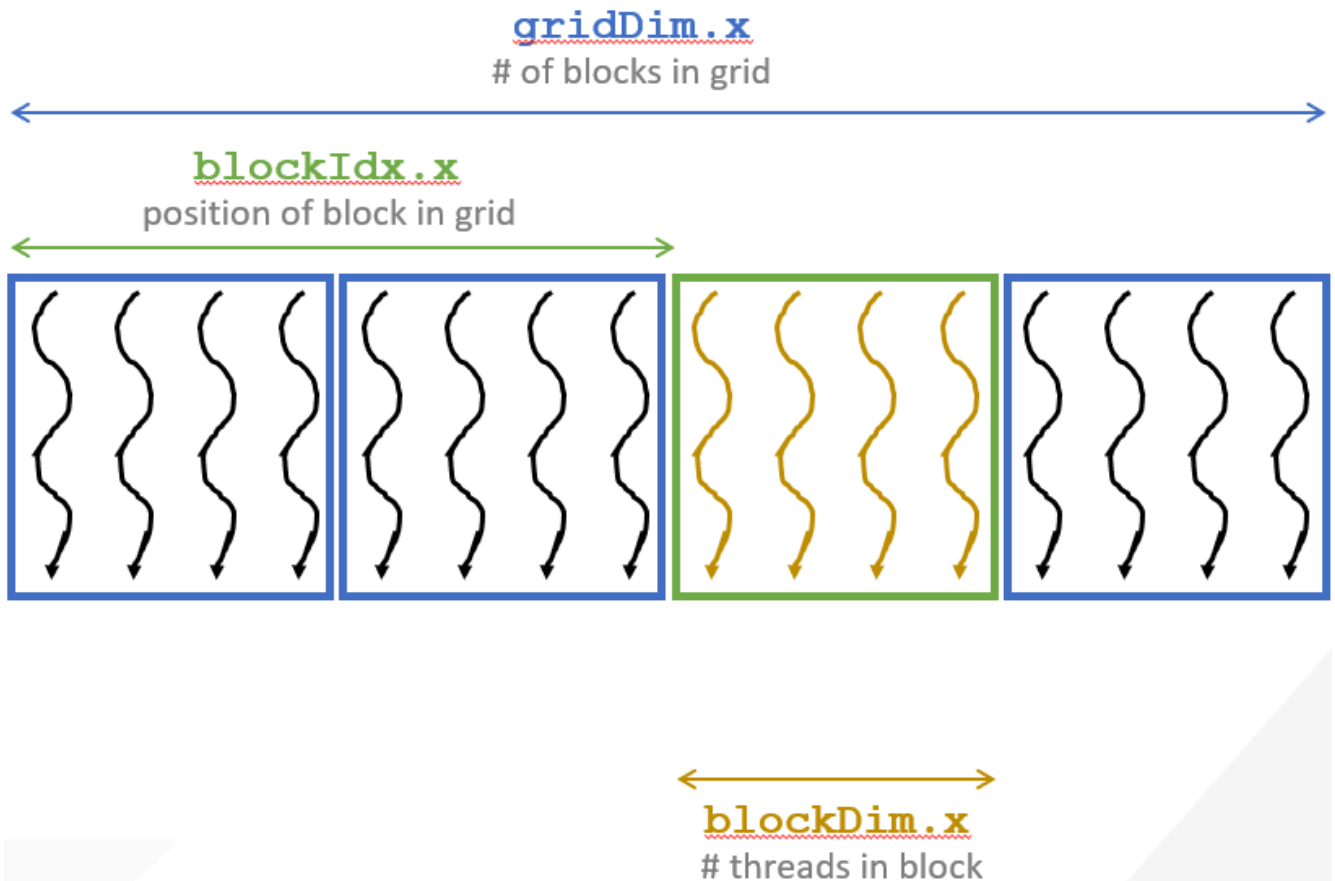


Рис. 39. blockDim — размерность блока

Индекс потока (Thread Index)

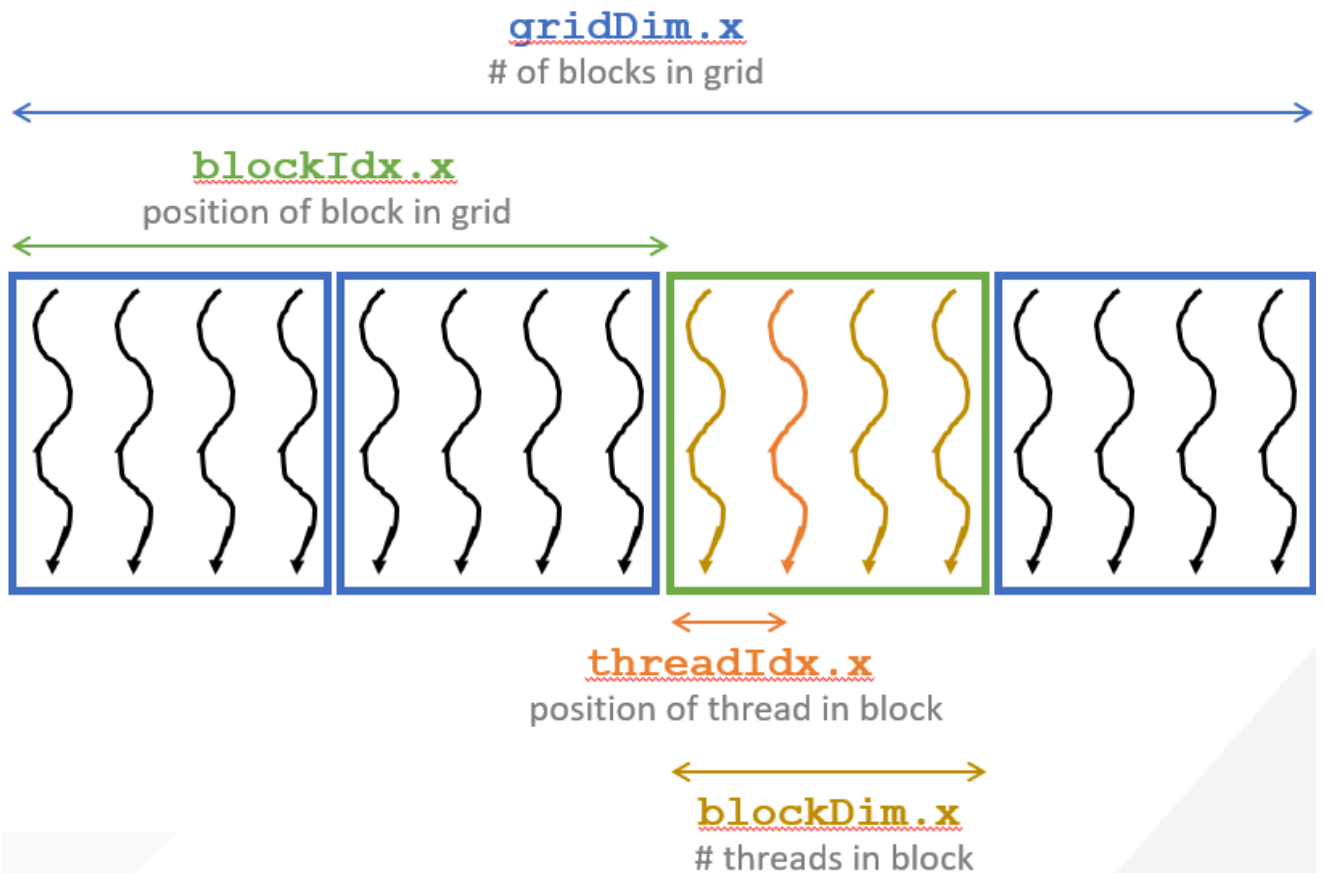


Рис. 40. threadIdx — индекс потока в блоке

Идентификатор кластера (Hopper+, 2022)



- `clusterIdx.x`, `clusterIdx.y`, `clusterIdx.z` — индекс кластера блоков
- `clusterDim.x`, `clusterDim.y`, `clusterDim.z` — размеры кластера
- Доступны только в ядрах с объявлением `__cluster_dims__`

```
__global__ void __cluster_dims__(2, 1, 1)
cluster_kernel(float* data, int N) {
    // Индекс кластера
    int cluster_x = clusterIdx.x;
    int cluster_y = clusterIdx.y;

    // Глобальный индекс блока с учётом кластера
    int global_block_x = clusterIdx.x * clusterDim.x + blockIdx.x;

    // ...
}
```

Вычисление глобального индекса потока



```cpp
// 1D случай (наиболее распространённый)
int i = blockIdx.x * blockDim.x + threadIdx.x;

// 2D случай (изображения, матрицы)
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
int idx = row * width + col;

// 3D случай (объёмные данные)
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
int z = blockIdx.z * blockDim.z + threadIdx.z;
int idx = x + y * width + z * width * height;
```

Специальные переменные (сводка)

Переменная	Тип	Описание
`gridDim`	`dim3`	Размеры решётки
`blockIdx`	`uint3`	Индекс блока в решётке
`blockDim`	`dim3`	Размеры блока
`threadIdx`	`uint3`	Индекс потока в блоке
`warpSize`	`int`	Размер варпа (всегда 32)
`clusterDim`	`dim3`	Размеры кластера (Hopper+)
`clusterIdx`	`uint3`	Индекс кластера (Hopper+)

ПАМЯТЬ УСТРОЙСТВА

Пример: преобразование RGB в оттенки серого



Рис. 41. Исходное RGB-изображение



Рис. 42. Результат преобразования в оттенки серого



- Подход к параллелизации: назначить один поток на каждый пиксель изображения для его преобразования.

Многомерные решётки (Multidimensional Grids)



- CUDA поддерживает многомерные решётки (до 3D).
  - Упрощает обработку многомерных данных.

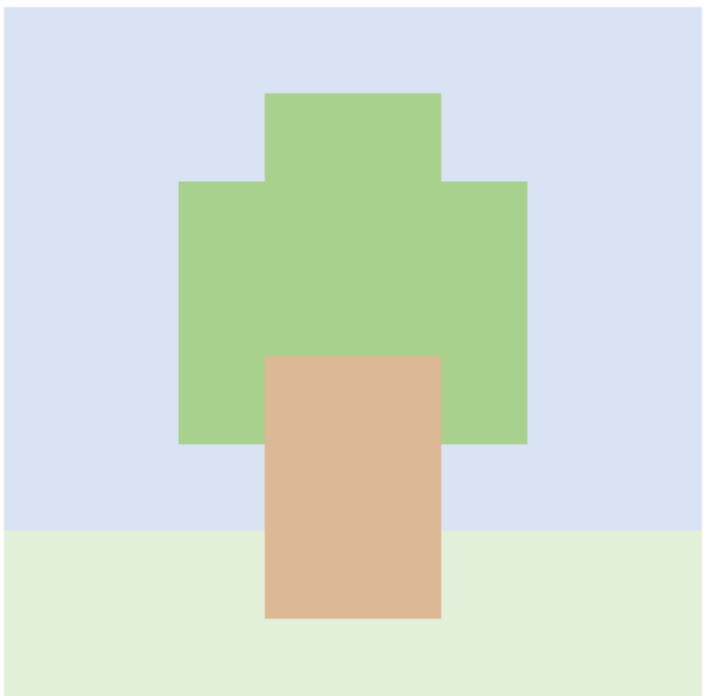


Рис. 43. Двумерное изображение как данные

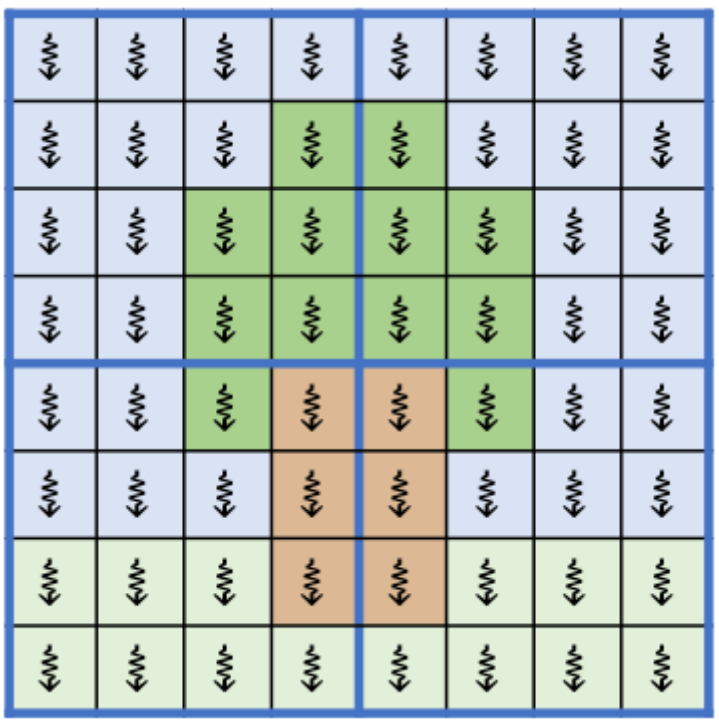


Рис. 44. Назначение блоков и потоков для 2D-данных

Настройка многомерных решёток



- Использование встроенного типа `dim3`.
  - Остальное то же самое.

```
dim3 numThreadsPerBlock(32, 32);
dim3 numBlocks((width + numThreadsPerBlock.x - 1) / numThreadsPerBlock.x,
               (height + numThreadsPerBlock.y - 1) / numThreadsPerBlock.y);
rgb2gray_kernel <<< numBlocks, numThreadsPerBlock >>>
               (red_d, green_d, blue_d, gray_d, width, height);
```

Многомерная индексация (Multidimensional Indexing)



- Встроенные переменные размерности и индекса имеют три компонента: x, y и z.

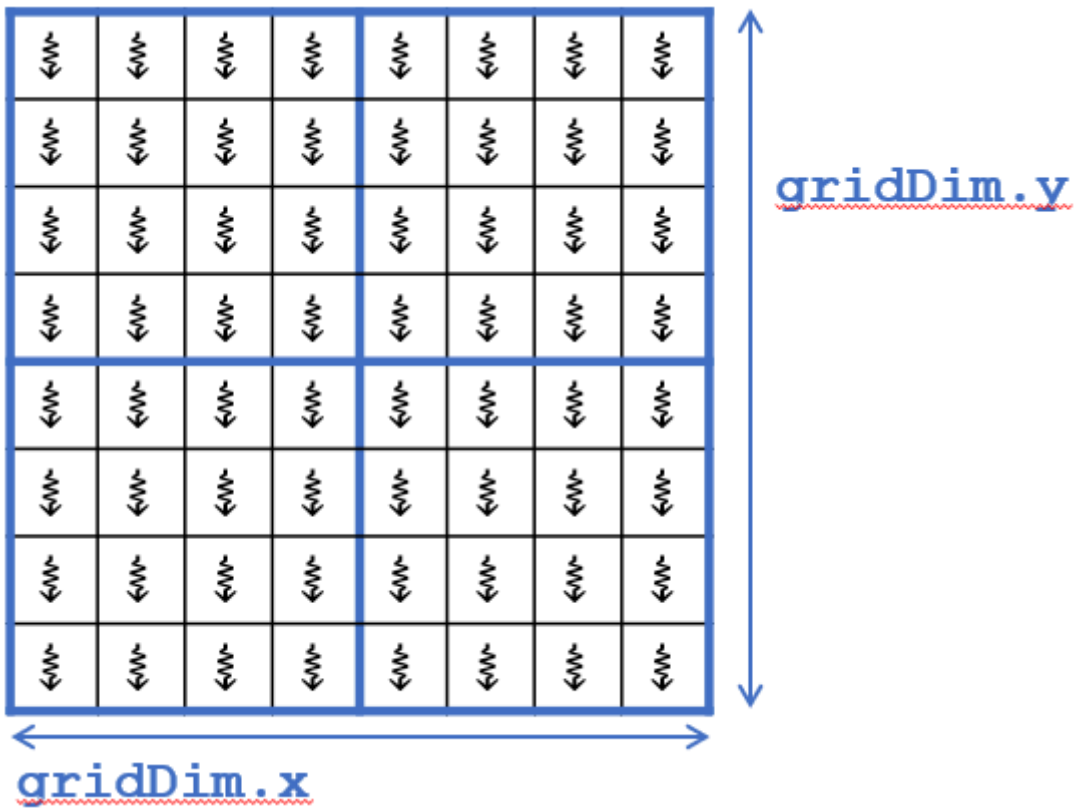


Рис. 45. Многомерная индексация: gridDim

Многомерная индексация (продолжение)

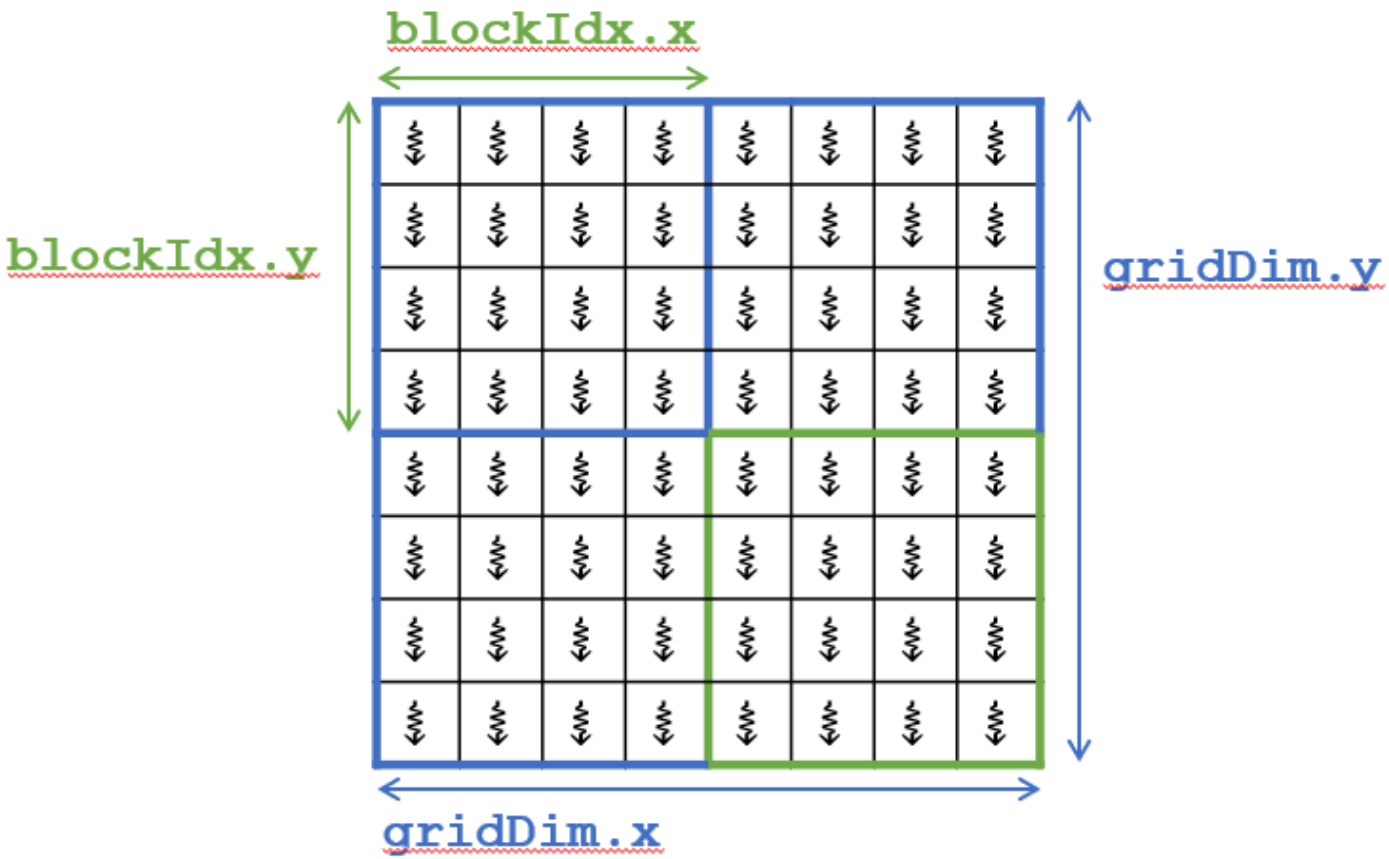


Рис. 46. Многомерная индексация: blockIdx

Многомерная индексация (продолжение)

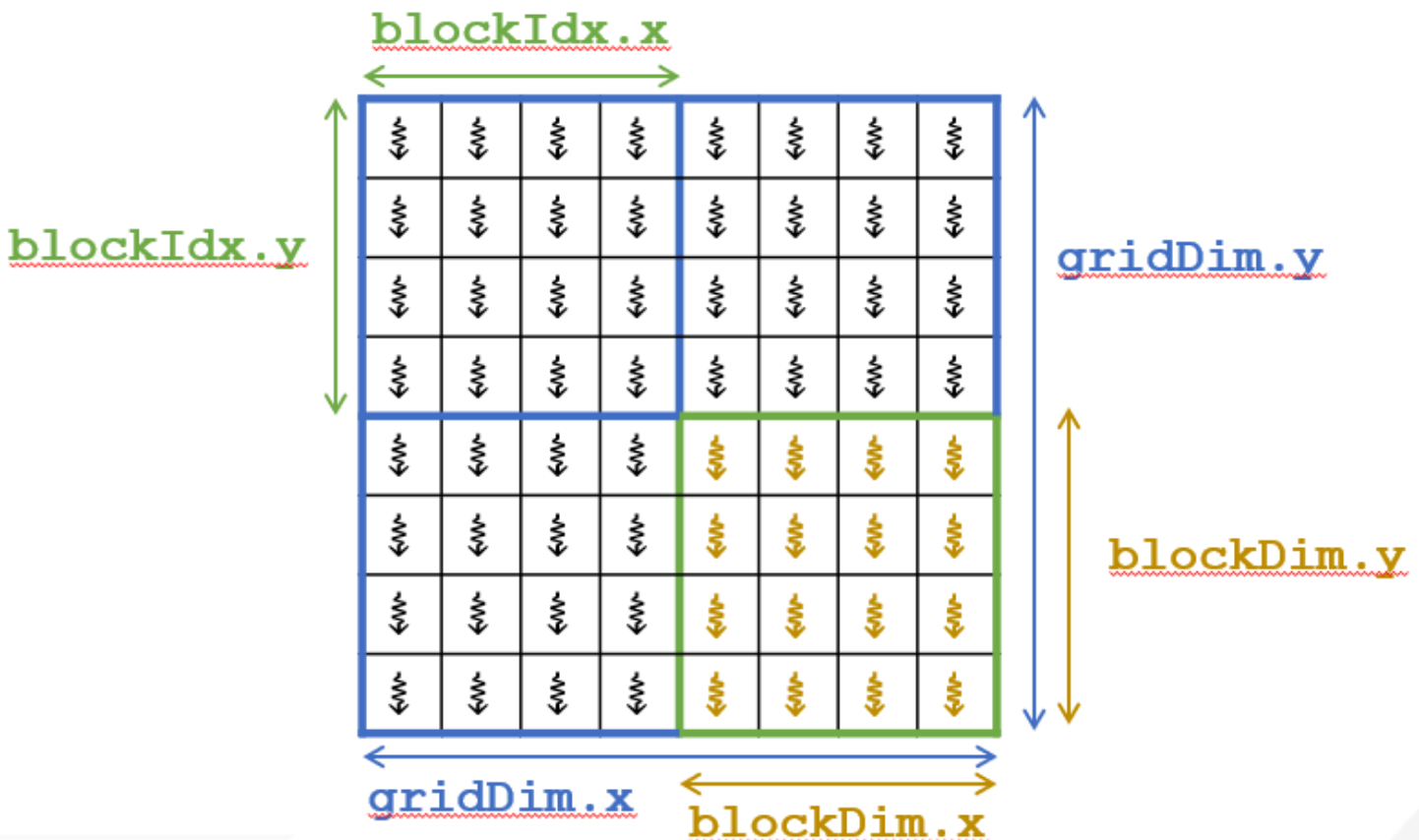


Рис. 47. Многомерная индексация: blockDim

Многомерная индексация (продолжение)

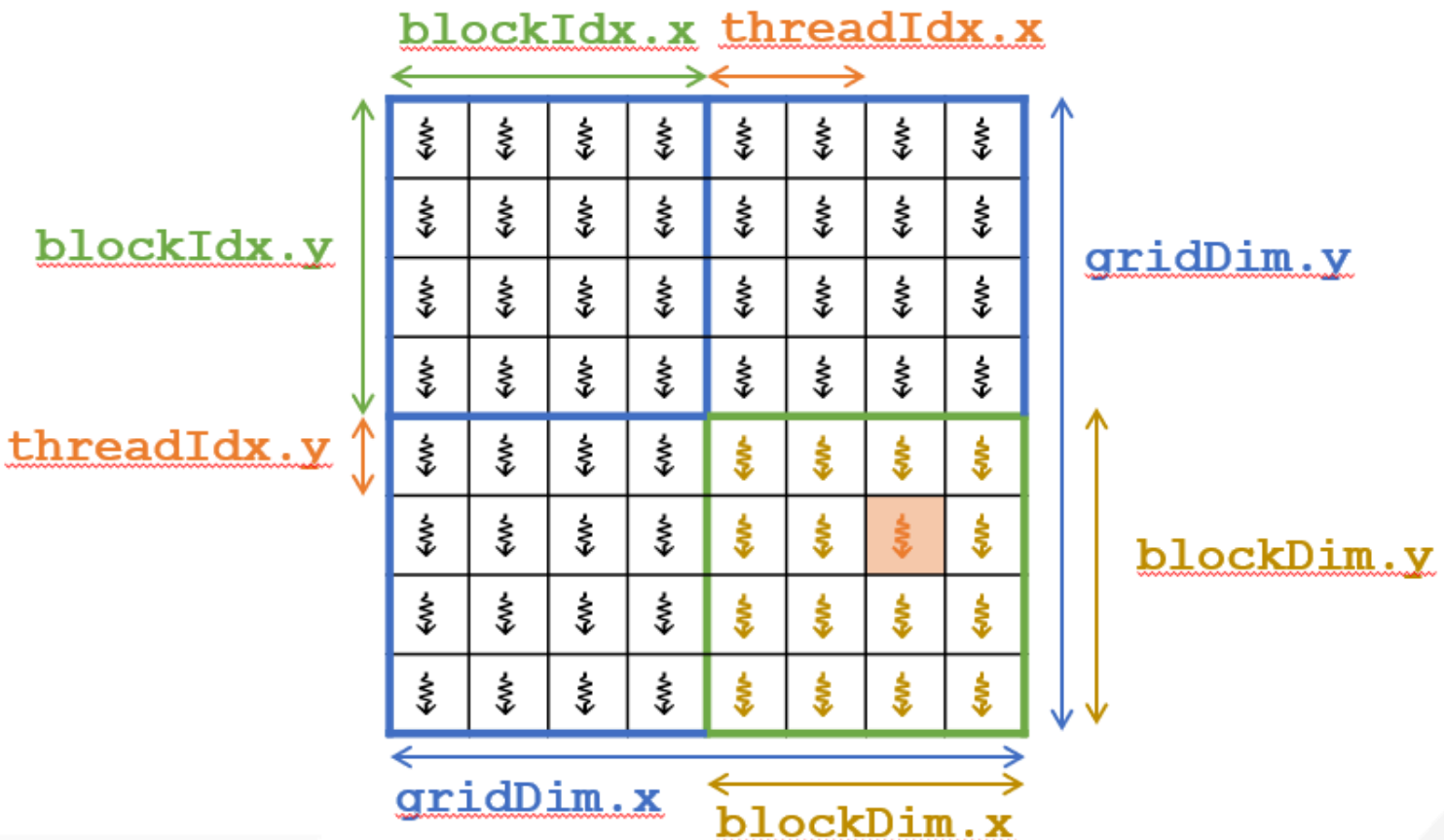


Рис. 48. Многомерная индексация: threadIdx

Пример многомерной индексации



```
__global__ void rgb2gray_kernel(unsigned char* red, unsigned char* green, unsigned char* blue,
                                unsigned char* gray, unsigned int width, unsigned int height) {
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;

    // Преобразование пикселя
    // ...
}
```

Размещение многомерных данных (Layout of Multidimensional Data)



- В C принято хранить данные в порядке следования строк (row major order).
  - Элементы в одной строке расположены в памяти непрерывно.

Logical view of data

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3

Actual layout in memory

0,0	0,1	0,2	0,3	1,0	1,1	1,2	1,3	2,0	2,1	2,2	2,3	3,0	3,1	3,2	3,3

Рис. 49. Логическое и фактическое размещение многомерных данных в памяти

Доступ к многомерным данным (Accessing Multidimensional Data)



- `index = row * width + col`

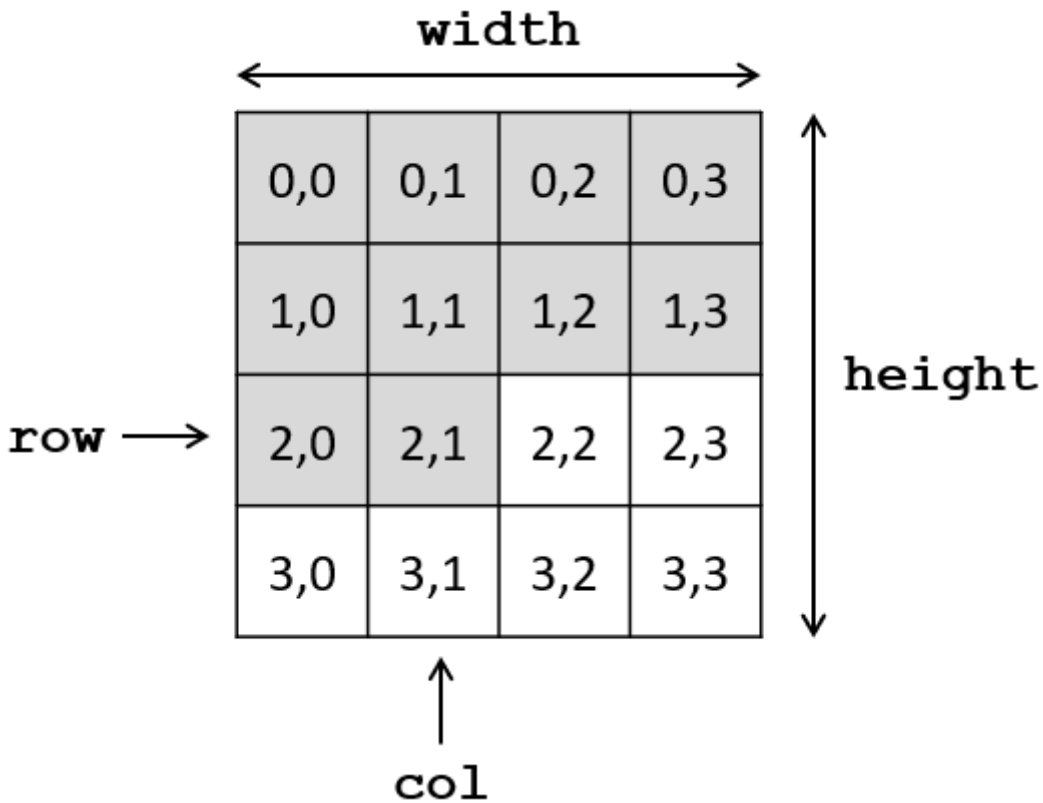


Рис. 50. Вычисление линейного индекса для 2D-данных

Пример доступа к многомерным данным



```
__global__ void rgb2gray_kernel(unsigned char* red, unsigned char* green, unsigned char* blue,
                                unsigned char* gray, unsigned int width, unsigned int height) {
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;

    // Преобразование пикселя
    gray[row * width + col] = red[row * width + col] * 3 / 10
                            + green[row * width + col] * 6 / 10
                            + blue[row * width + col] * 1 / 10;
}
```

Граничные условия (Boundary Conditions)

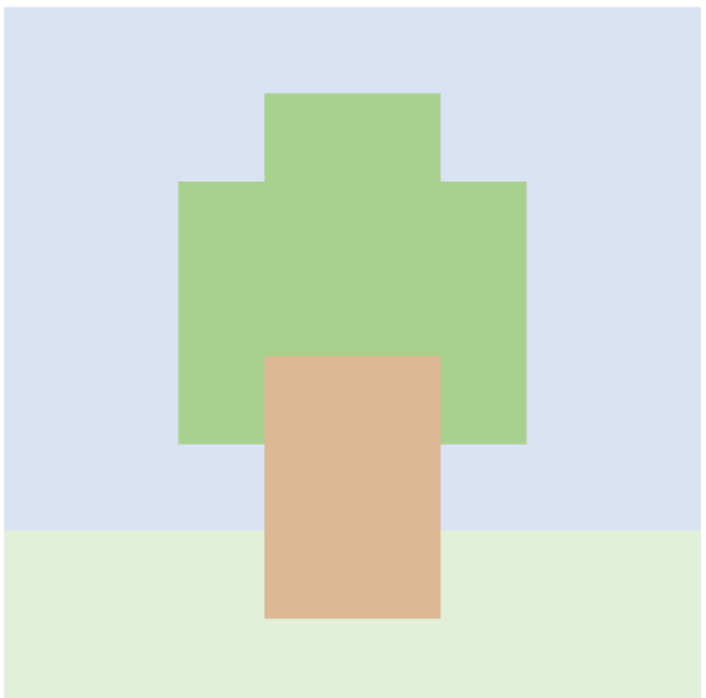


Рис. 51. Исходное изображение



Рис. 52. Назначение блоков и потоков



- Может быть больше потоков, чем пикселей в обоих измерениях.

Граничные условия (Код)



```
__global__ void rgb2gray_kernel(unsigned char* red, unsigned char* green, unsigned char* blue,
                                unsigned char* gray, unsigned int width, unsigned int height) {
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;

    // Преобразование пикселя
    if (row < height && col < width) {
        gray[row * width + col] = red[row * width + col] * 3 / 10
                                + green[row * width + col] * 6 / 10
                                + blue[row * width + col] / 10;
    }
}
```

TMA: Tensor Memory Accelerator (Hopper+, 2022)



- Аппаратный ускоритель перемещения данных
- Асинхронное копирование глобальная ↔ разделяемая память
  - `cp.async` — не блокирует вычисления
  - TMA (Tensor Memory Accelerator) — аппаратный ускоритель
- Поддержка многомерных трансформаций (transpose, tile)
- Снижает нагрузку на вычислительные ядра

```
// Пример использования TMA через async copy
__global__ void tma_kernel(float* global_data, int N) {
    __shared__ float shared_data[256];

    // Асинхронная загрузка через TMA
    cuda::memcpy_async(shared_data, global_data,
                       cuda::aligned_size_t<16>(256 * sizeof(float)));
    cuda::barrier<cuda::thread_scope_block> bar;
    bar.arrive_and_wait();
```

```
    // Вычисления с данными в shared memory
    // ...
}
```

Distributed Shared Memory (Hopper+, 2022)



- Доступ к разделяемой памяти блоков в том же Thread Block Cluster
- Прямой обмен данными между соседними SM без глобальной памяти
- Критично для масштабируемых алгоритмов

```
__global__ void __cluster_dims__(2, 1, 1)
distributed_sm_kernel(float* data, int N) {
    __shared__ float local_data[256];

    // Доступ к shared memory соседнего блока в кластере
    float* remote_sm = cluster::map_shared_rank(local_data, 1);

    // Копирование данных из remote shared memory
    cluster::memcpy_async(remote_sm, local_data, sizeof(local_data));
    cluster::sync();

    // Вычисления с локальными и удалёнными данными
    // ...
}
```



```
]

// Async copy (Volta+, улучшен в Hopper)
__shared__ float tile[BLOCK_SIZE];
int tid = threadIdx.x;
// Асинхронная загрузка в shared memory
cp.async.ca.shared.global [shared_mem], [global_mem], 16;
cp.async.commit_group;
cp.async.wait_group 0;
__syncthreads();
// Теперь данные в shared memory готовы к использованию
```



ПРИМЕР ПРОСТЕЙШЕЙ ПРОГРАММЫ

Пример: размытие изображения (Blur)



Рис. 53. Исходное изображение для размытия



Рис. 54. Результат размытия изображения



- Выходной пиксель является средним значением соответствующего входного пикселя и пикселей вокруг него

Пример: размытие изображения (Подход)

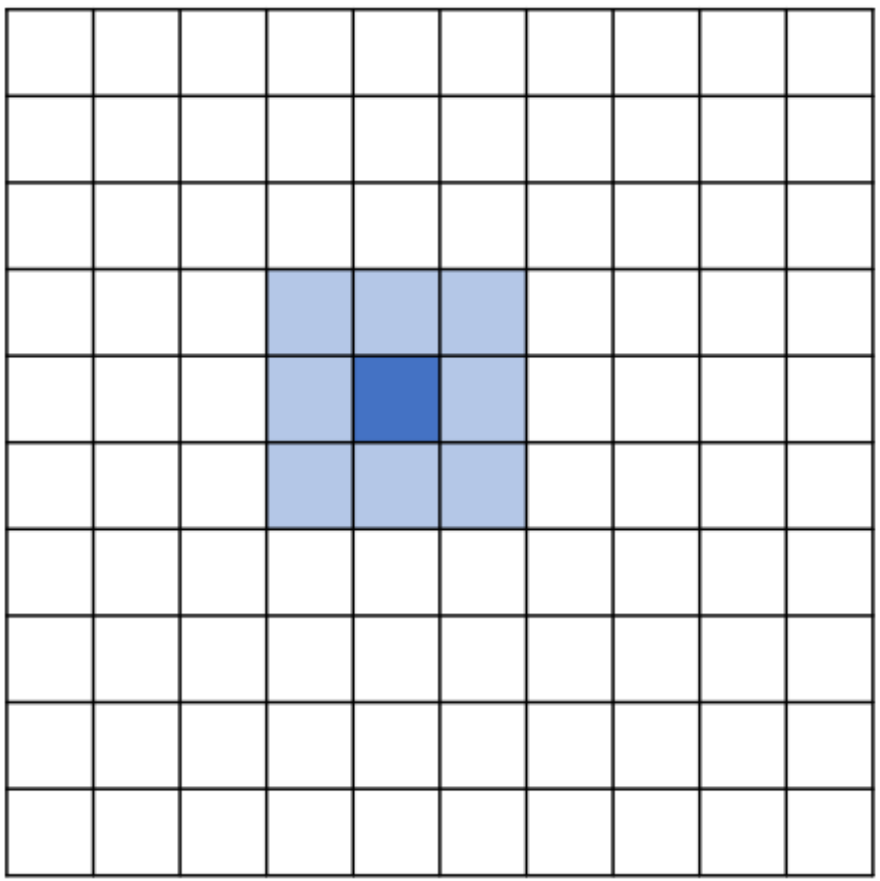


Рис. 55. Входное изображение и окно размытия

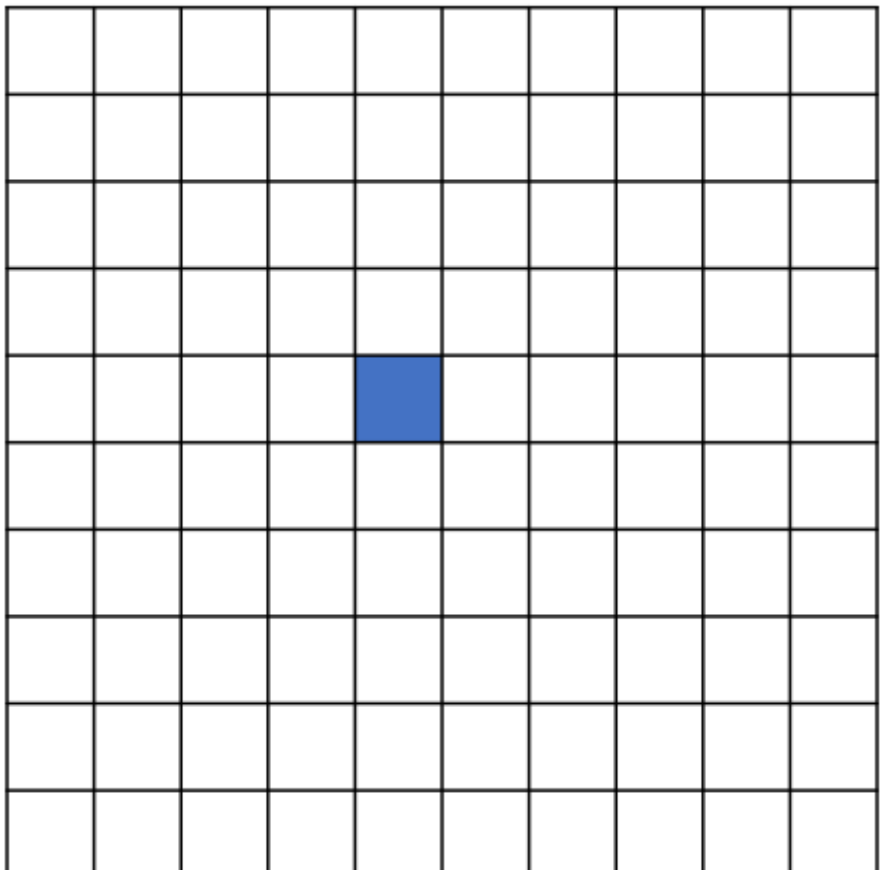


Рис. 56. Выходное размытое изображение



- Подход к параллелизации: назначить один поток на каждый выходной пиксель и заставить его читать несколько входных пикселей

Пример: размытие изображения (Код)



```
__global__ void blur_kernel(unsigned char* image, unsigned char* blurred,
                            unsigned int width, unsigned int height) {
    int outRow = blockIdx.y * blockDim.y + threadIdx.y;
    int outCol = blockIdx.x * blockDim.x + threadIdx.x;

    if (outRow < height && outCol < width) {
        unsigned int average = 0;
        for(int inRow = outRow - BLUR_SIZE; inRow < outRow + BLUR_SIZE + 1; ++inRow) {
            for(int inCol = outCol - BLUR_SIZE; inCol < outCol + BLUR_SIZE + 1; ++inCol) {
                average += image[inRow * width + inCol];
            }
        }
        blurred[outRow * width + outCol] =
                  (unsigned char)(average / ((2 * BLUR_SIZE + 1) * (2 * BLUR_SIZE + 1)));
    }
}
```

Граничные условия (Размытие)

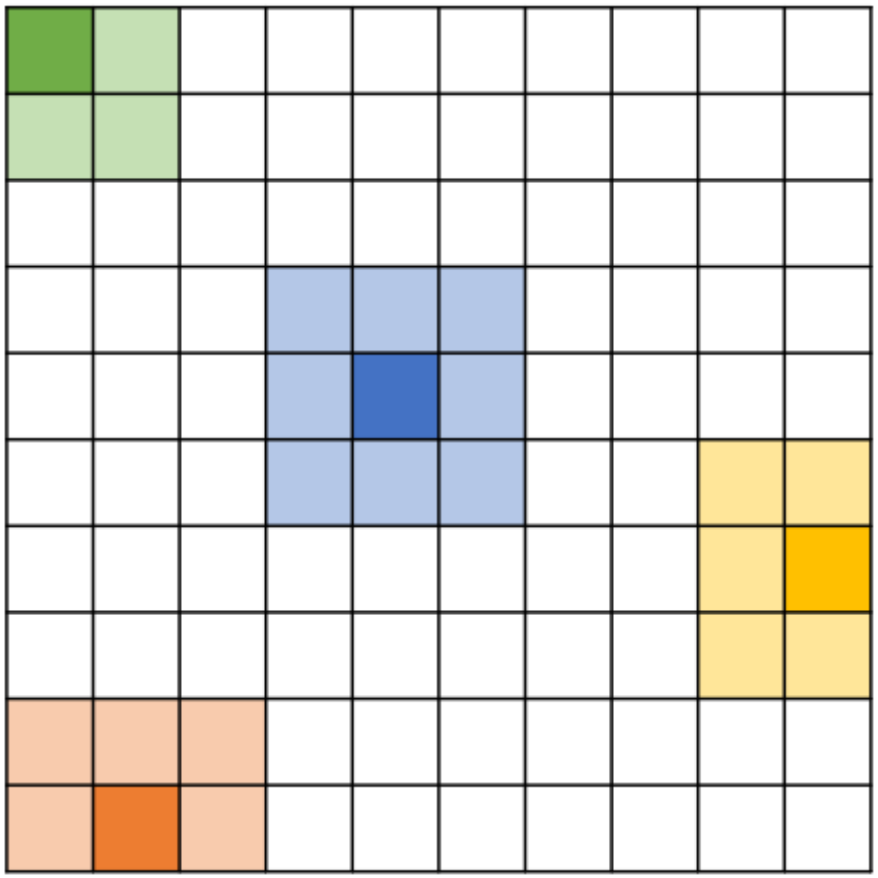


Рис. 57. Входное изображение с граничными пикселями

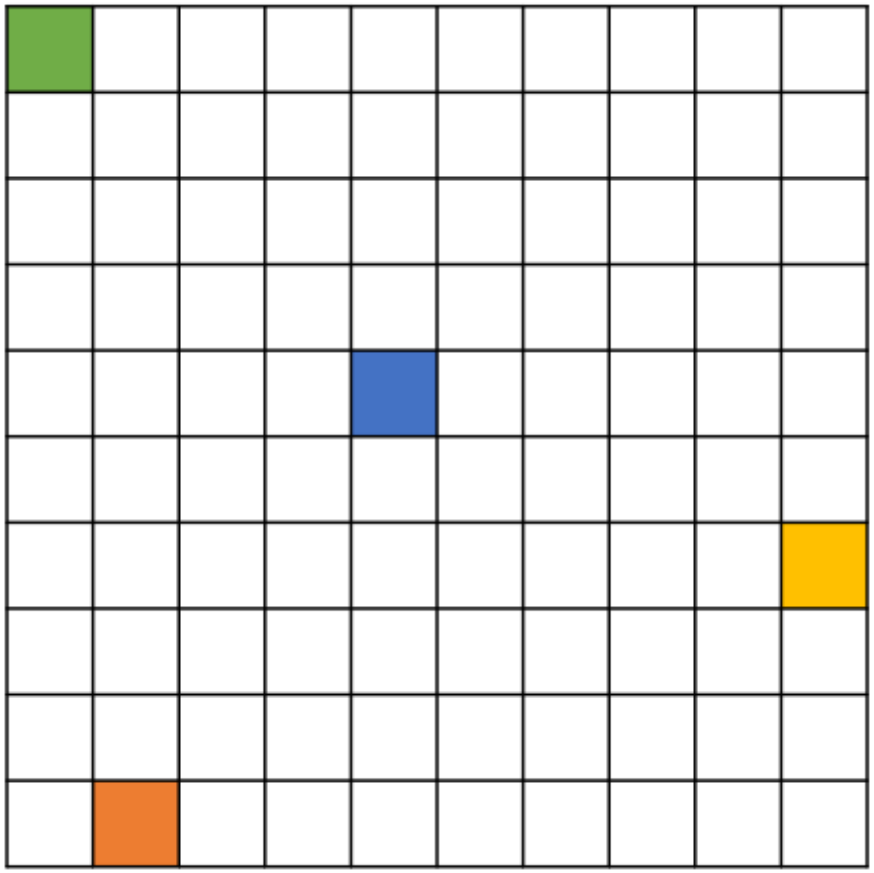


Рис. 58. Выходное размытое изображение с учётом границ



- Даже если выходной пиксель находится в пределах, может происходить обращение к входному пикселю за пределами

Граничные условия (Размытие, Код)



```
// ...
unsigned int average = 0;
for(int inRow = outRow - BLUR_SIZE; inRow < outRow + BLUR_SIZE + 1; ++inRow) {
    for(int inCol = outCol - BLUR_SIZE; inCol < outCol + BLUR_SIZE + 1; ++inCol) {
        if(inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
            average += image[inRow * width + inCol];
        }
    }
}
// ...
```

Пример: умножение матриц (Matrix-Matrix Multiplication)

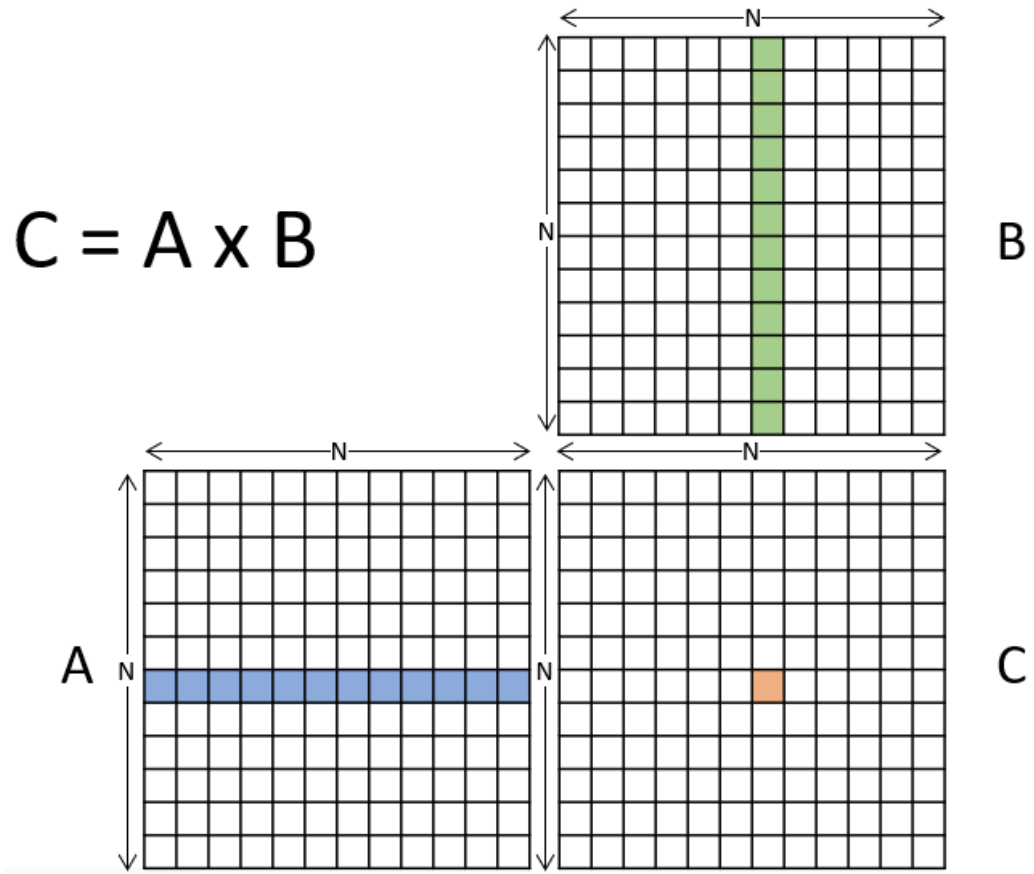


Рис. 59. Умножение матриц: задача

Пример: умножение матриц (Подход)

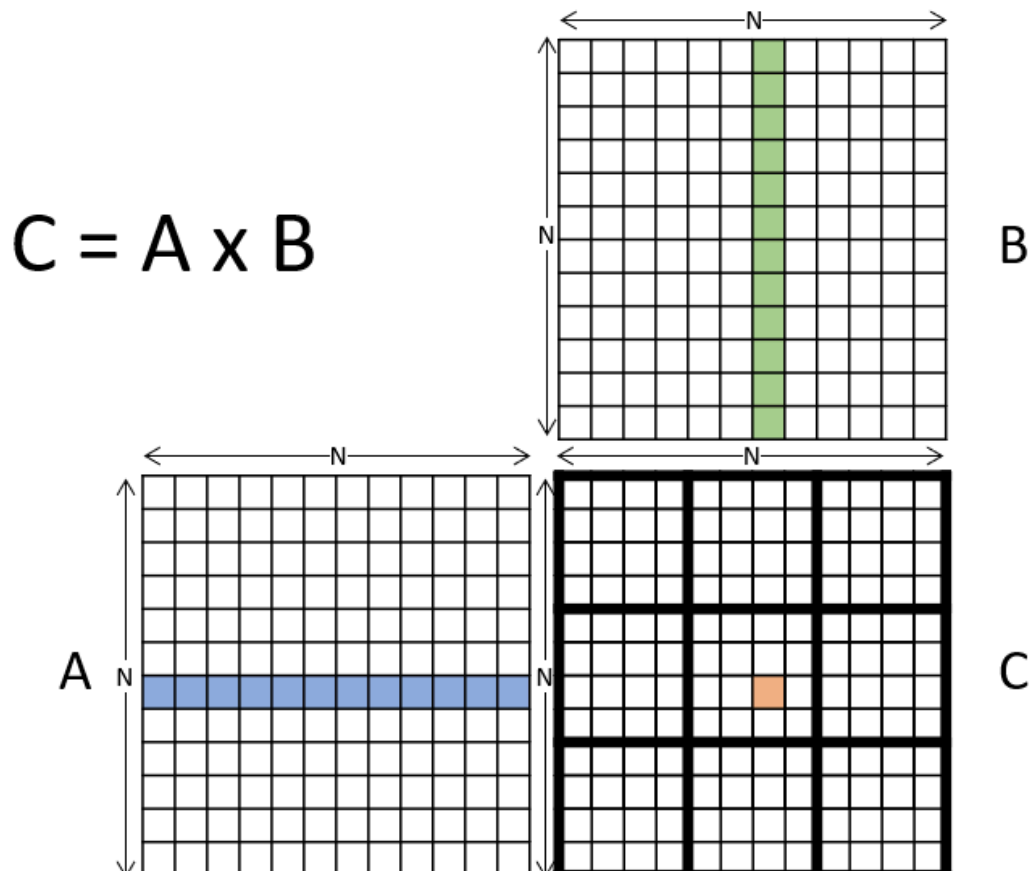


Рис. 60. Умножение матриц: подход к параллелизации

Пример: умножения матриц (Код, базовый)



```
__global__ void mm_kernel(float* A, float* B, float* C, unsigned int N) {
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;

    float sum = 0.0f;
    for(unsigned int i = 0; i < N; ++i) {
        sum += A[row * N + i] * B[i * N + col];
    }
    C[row * N + col] = sum;
}
```

Пример: умножение матриц (Визуализация)

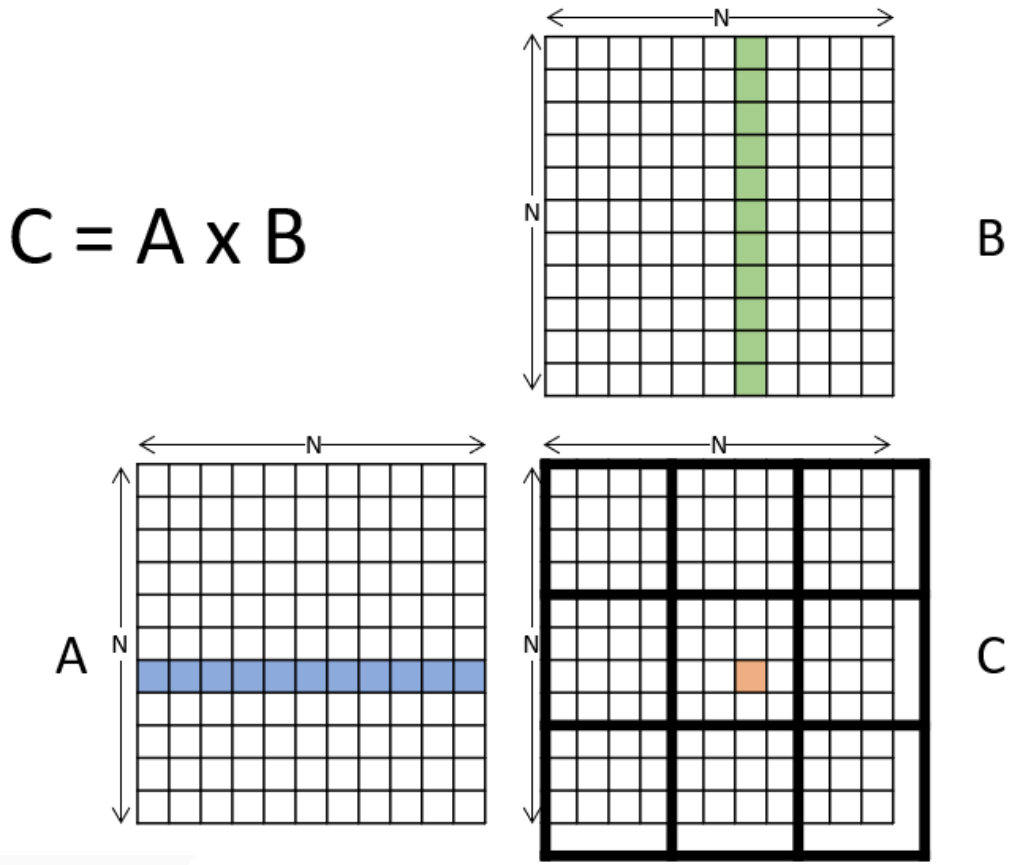


Рис. 61. Умножение матриц: визуализация вычислений

Пример: умножение матриц (Визуализация)

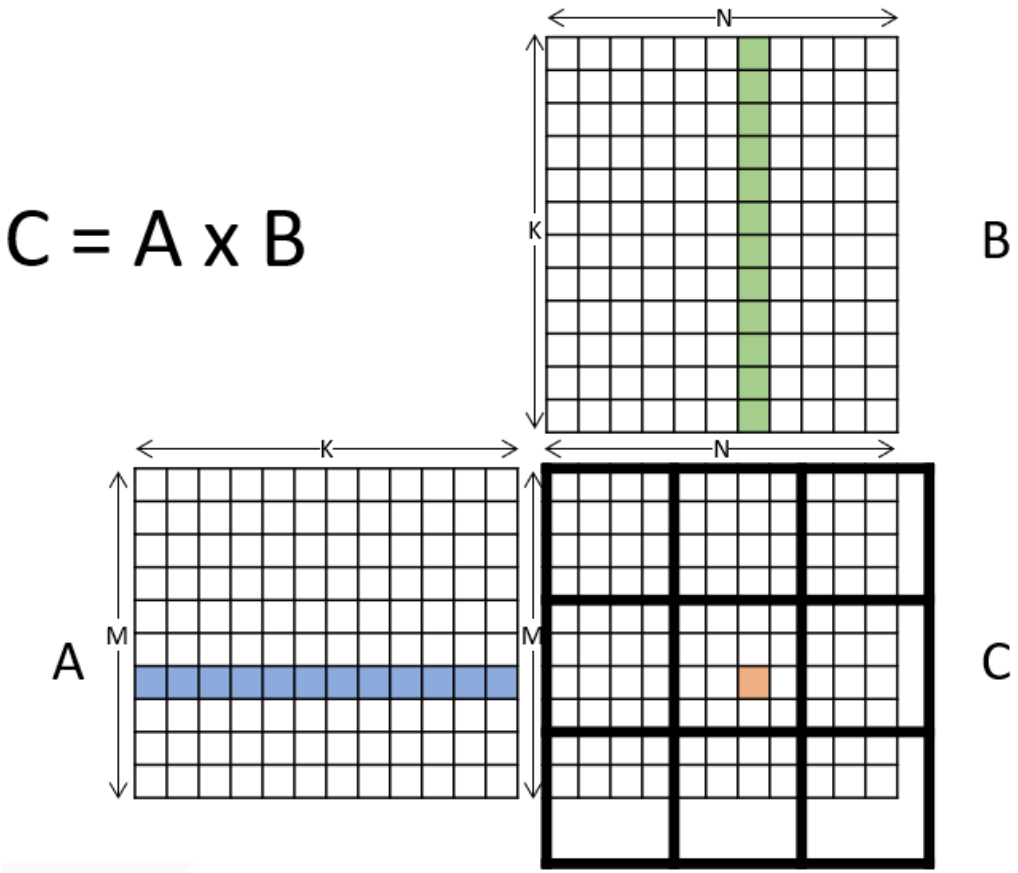


Рис. 62. Умножение матриц: итоговый результат

Современные подходы к умножению матриц



- **Tensor Cores**: аппаратное умножение матриц 16x16 (Hopper) или 32x32 (Blackwell)
- **WMMA/WGMMA API**: программный интерфейс к Tensor Cores
- **Cutlass**: библиотека NVIDIA для высокопроизводительных матричных операций
- **cuBLAS**: оптимизированная библиотека линейной алгебры

```
// Пример использования WMMA (Warp Matrix Multiply-Accumulate)
#include <mma.h>
using namespace nvcuda;

__global__ void wmma_kernel(half* A, half* B, float* C, int N) {
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    wmma::fill_fragment(c_frag, 0.0f);
    wmma::load_matrix_sync(a_frag, A, N);
    wmma::load_matrix_sync(b_frag, B, N);
    wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
```

```
    wmma::store_matrix_sync(C, c_frag, N, wmma::mem_row_major);
}
```

Интересные статьи



- История видеопроцессоров:
  - 1976 — 1995 годы
  - 3Dfx Voodoo
  - Консолидация рынка, начало эпохи конкуренции Nvidia и ATI
  - Появление General Purpose GPU
- Графические войны 1: лагающее пиксельное XX столетие
- Ретроспектива: с чего начиналась 3D-графика на ПК. Часть 1: 3dfx Interactive — взлёт
- Простая самодельная VGA видеокарта
- CUDA Documentation (2026)
- NVIDIA CUTLASS Library
- NVIDIA Developer Blog